

基于启发式算法的烟幕干扰弹投放策略优化模型

摘 要

在导弹来袭过程中，通过投放烟幕干扰弹遮蔽真目标，因此针对不同情况，分别设计烟幕干扰弹的投放策略，使得多枚烟幕干扰弹对真目标的有效遮蔽时间尽可能长。

针对问题一，计算烟幕干扰弹对 M1 的有效遮蔽时长。首先，构建无人机、导弹、真假目标的三维空间模型。其次，根据动力学原理，建立研究对象的空间位置解析表达式。然后，运用解析几何方法构建从导弹到真实圆柱目标各点的视线集合。确立有效遮蔽的判定准则为：烟幕云团必须同时遮蔽所有关键视线。最后，通过几何分析，明确合理的采样方式将该条件量化为云团球心到视线最大距离不大于 10 米。可视化球心点到视线最大距离随时间 t 的曲线确定有效遮蔽时间。有效遮蔽结果为 1.3901s。

问题二：确定 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点。首先，确定目标优化模型，包含飞行方向等四维决策变量，多约束条件以及目标函数。其次，建立无人机 FY1 烟幕弹投放策略优化模型，考虑到梯度下降这种传统优化方法易陷入局部最优，使用改进的遗传算法进行参数更新优化。通过锦标赛策略选择与高斯变异操作实现全局与局部搜索最优。然后，综合考虑时间与精度固定搜索步长以便计算时长。最终有效遮蔽时长约为 4.94s。

问题三：构建单机多弹协同优化模型。在问题二的基础上，增加了相邻干扰弹的投放间隔等决策变量，以及时间间隔约束。首先，采用增量参数表示法，用增量约束相邻干扰弹间隔，间接保证了投放顺序。其次，考虑云团匀速下沉与 20s 有效时长，采用“任一云团遮蔽关键点即可”的联合判定准则，实现多弹协同延长遮蔽时长。最后采用基于 best1bin 策略与贪婪选择改进的差分进化策略求解最优情况，最终有效遮蔽时长为 6.17s，记录在表 result1.xlsx 中。

问题四：构建多无人机协同优化模型。在问题二的基础上，拓展了多机协调干扰的情况，对于每架无人机都拥有单独的决策向量。利用遗传算法以所有无人机投放的烟幕弹的有效干扰时间的并集即总遮蔽时间作为适应度，在并行计算环境下求解不断迭代优化，求解最优烟幕弹投放策略，最终求解得到的总有效遮蔽时长为 8.58 s。同时对单无人机的遮蔽时间贡献进行分析，得到 FY1 独立遮蔽时间: 4.96 s, FY2 独立遮蔽时间: 3.62 s, FY3 独立遮蔽时间: 0.00 s。最后将最优烟幕弹投放策略的相关参数填入表 result2.xlsx 中。

问题五：针对问题五“多机多弹投放干扰 3 枚来袭导弹”的协同优化问题，为避免直接使用遗传算法陷入维度灾难，提出“贪心算法生成初始解+遗传算法优化”的求解框架。首先定义无人机集合、导弹集合及指示变量，新增“每架无人机至多干扰 1 枚导弹”“每枚导弹至少被 1 架无人机干扰”约束；采用贪心算法生成“任务分配矩阵+弹药数量向量”初始解，再结合遗传算法将问题转化为多个问题 3（单机多弹）和问题 4（多机单弹）的优化，最大化多无人机-多烟幕弹对 3 枚导弹的总有效遮蔽时间。最终计算的有效遮蔽时长为 22.9s，并最优烟幕弹投放策略参数填入表 result3.xlsx 中。

关键词：烟幕干扰弹 有效遮蔽 目标优化 多机协同

一、 问题重述

烟幕干扰弹形成烟幕或气溶胶云团，可以形成遮蔽，干扰敌方导弹，具有成本低、效费比高等优点。合适的烟幕干扰弹的投放策略能精确控制烟幕干扰弹到达预定位置，并控制起爆时间以及烟幕云团下沉速度。为防止地点暴露，采用假目标诱导导弹，并利用烟幕云团遮蔽真目标，真目标为半径 7 m、高 10 m 的圆柱形。

现有来袭空地导弹 3 枚分别位于三处不同位置，导弹飞行速度为 300m/s，直指假目标。现有五架无人机负责投放烟幕干扰弹对真目标进行烟幕干扰。每架无人机初始位置信息已知，无人机受领任务后，以 70~140 m/s 的速度等高度匀速直线飞行，并在合适位置投放干扰弹，每架无人机投放两枚烟幕干扰弹至少间隔 1 s。干扰弹在重力作用下运动一段时间后，爆炸形成云团，云团以 3 m/s 的速度匀速下沉，有效时间 20s。

根据以上信息，解决以下问题：

问题一，无人机 FY1 以 120 m/s 的速度朝向假目标方向飞行，受领任务 1.5 s 后即投放 1 枚烟幕干扰弹对 MI 进行干扰，间隔 3.6 s 后起爆。计算有效遮蔽时长。

问题二，在问题一的情况下，为使有效遮蔽时长尽可能长，求最佳的无人机 FY1 飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点。

问题三，在问题二的基础上，共投放 3 枚烟幕干扰弹，确定最佳投放策略，此时需要注意两枚烟幕干扰弹至少间隔 1 s。

问题四，多机单弹投放，确定三架无人机各投放一枚干扰弹的投放策略，三台无人机的速度、方向可以不同，但是一旦确定就不再调整。

问题五，多机多弹投放，5 架无人机，每架无人机至多投放 3 枚烟幕干扰弹，实施对 M1、M2、M3 等 3 枚来袭导弹的干扰，确定烟幕干扰弹的投放策略。

二、 问题分析

2.1 问题一的分析

对于问题一，使用无人机 FY1 对导弹进行干扰，并给定了初始速度及方向。同时，干扰弹投放时间与起爆时间固定。首先，分析导弹、无人机与烟幕干扰弹的运动情况，根据运动学原理建立三维运动轨迹模型，无人机在一定高度做匀速直线运动，导弹朝假目标做匀速直线运动，干扰弹先做平抛运动，爆炸后做匀速竖直下落。其次，根据解析几何构建导弹与真实目标的视线数学表达公式。最后，为计算有效时间遮挡，根据点与直线距离公式建立有效遮挡判定标准与数学表达模型，构建合理的采样策略，设置合适时间步长，兼顾精度与效率。

2.2 问题二的分析

对于问题二，在问题一基础上需要同时优化速度、方向角、投放时间、起爆延迟 4 个决策变量，使有效时间尽量长。由于这些变量相互影响，形成复杂的非线性关系，传统局部优化算法易陷入局部最优，而遗传算法通过维持种群多样性、实现基因交换，能在更大范围内搜索全局最优解。因此，本题建立基于遗传算法的无人机 FY1 烟幕弹投放策略优化模型，结合多个几何与物理条件约束，获得最优投放策略。

2.3 问题三的分析

对于问题三，在问题二的基础上，改变了无人机 FY1 投放烟幕干扰弹的数量，由

于每架无人机投放两枚烟幕干扰弹至少间隔 1 s，在问题二的基础上添加时间间隔约束。此外，本题共建立八个决策变量，对于高纬度约束优化，本团队建立基于差分进化的投放策略优化模型，覆盖高维解空间得到最优解。

2.4 问题四的分析

对于问题四，当无人机数量增加至 3 架，且可投放三枚烟幕干扰弹，同时没有时间约束时，这等价于在问题二的基础上增加了 8 个决策变量，使决策变量总数达到 12 个。也就是说，每架无人机拥有 4 个决策变量，并且未增加新的约束条件。因此，可沿用问题二基于遗传算法的投放策略优化模型，以获得最优投放策略。

2.5 问题五的分析

针对问题五“多机多弹投放干扰 3 枚来袭导弹”的协同优化问题，为避免直接使用遗传算法陷入维度灾难，提出“贪心算法生成初始解+遗传算法优化”的求解框架。首先定义无人机集合、导弹集合及指示变量，新增“每架无人机至多干扰 1 枚导弹”“每枚导弹至少被 1 架无人机干扰”约束；采用贪心算法生成“任务分配矩阵+弹药数量向量”初始解，再结合遗传算法将问题转化为多个问题 3（单机多弹）和问题 4（多机单弹）的优化，最大化多无人机-多烟幕弹对 3 枚导弹的总有效遮蔽时间。

2.6 总思路图

根据以上分析，问题一到问题四的总架构图如下所示：

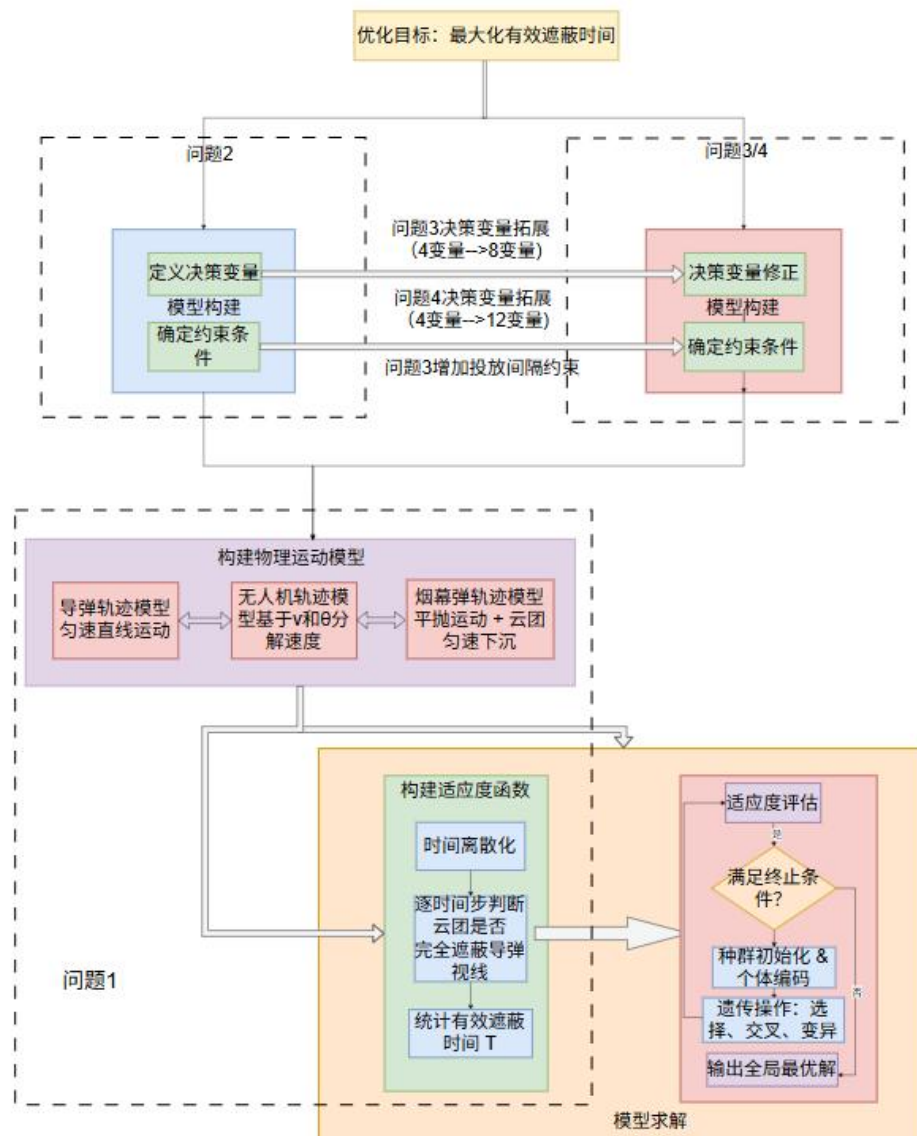


图 1 思路架构图

三、 模型假设

1. 假设导弹直线运动。
2. 假设烟幕干扰弹不受水平空气阻力作用。
3. 假设导弹、无人机可看作质点。
4. 假设无人机续航能力完全足够。
5. 假设烟幕弹起爆形成云团的时间极短，可完全忽略。
6. 不考虑风对云团的影响。
7. 假设投放烟雾弹时，能量没有损失。

四、 符号说明

符号	说明	单位
----	----	----

v_M	导弹飞行速度	m/s
v_s	烟幕云团下沉速度	m/s
t	时间变量	s
t_s	烟幕干扰弹起爆时间	s
f	无人机的位置	/
s	烟幕弹的位置	/
δ	烟幕弹延迟爆炸时间	s
v_F	无人机飞行速度	m/s
g	重力加速度	m/s^2
t_d	干扰弹投放时间	s
m	导弹在时刻 t 的位置	/
T_e	有效遮蔽时间	s
α	无人机飞行方向与轴夹角	rad
β	投放烟幕干扰弹的时间间隔	s

五、模型的建立与求解

5.1 问题一模型的建立与求解

模型展示：将导弹看作质点，则导弹看向真实目标圆柱体上的点的连线为导弹的视线。无人机在一定高度做匀速运动。

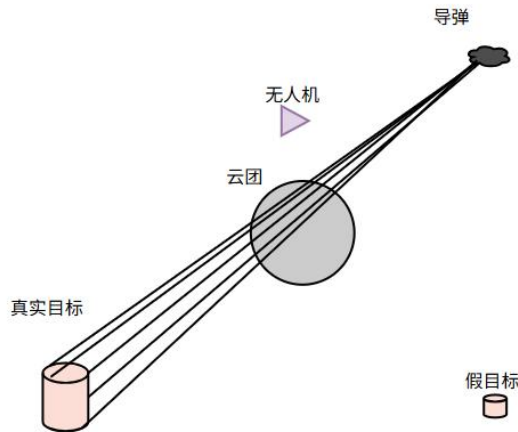


图 1 导弹视线模拟

5.1.1 三维运动轨迹模型的建立

导弹运动轨迹模型的确定

导弹从初始点出发，方向直指假目标，导弹飞行速度为 300 m/s。由于导弹在此过程中不受空气阻力等的影响，始终以匀速直线运动飞行。根据运动学原理，导弹飞行方向的单位向量为：

$$n = \frac{-\mathbf{m}_1}{\|\mathbf{m}_1\|} \quad (1)$$

根据单位向量得到导弹朝向与 x 轴的夹角 $\sin \gamma, \cos \gamma$ 值。则 M 导弹轨迹表示为：

$$\begin{aligned} \mathbf{m} &= \mathbf{m}_0 + \mathbf{v}_{M1} \times n \cdot t \\ \mathbf{m}_0 &= (20000, 0, 2000) \end{aligned} \quad (2)$$

FY1 无人机运动轨迹模型的确定

题目规定无人机高度匀速直线飞行，因此，z 轴方向速度始终为零，x 轴方向以 120m/s 的速度匀速飞行，y 轴方向速度为 0。因此 FY1 无人机运动轨迹方程表示为：

$$\begin{aligned} \mathbf{f} &= \mathbf{f}_0^T - \mathbf{v}_{F1} t \\ \mathbf{f}_0 &= (17800, 0, 1800)^T \\ \mathbf{v}_{F1} &= (120, 0, 0)^T \end{aligned} \quad (3)$$

当 $t = 1.5s$ 时，分别求出 M1 与 FY1 坐标： $\mathbf{m}_1$ 、 $\mathbf{f}_1$

烟幕弹运动轨迹模型的确定

烟幕弹投放时刻的位置即初始位置为此时无人机位置 $\mathbf{f}_1$ ，初期，由于受到重力作用，烟雾弹的运动轨迹为平抛运动曲线：

$$\mathbf{s} = \mathbf{f}_0^T - \left(120, 0, \frac{1}{2} g(t-1.5)^2 \right)^T \quad (4)$$

间隔 $\Delta t = 3.6s$ 后，此时 M 与烟雾弹坐标分别为 $\mathbf{m}_2$ 、 $\mathbf{s}_1$

云团运动轨迹模型的确定

烟雾弹爆炸后瞬时形成初始球状云团，云团的中心为 $\mathbf{s}_1$ ，此后，云团水平方向速度为零，竖直方向以 3m/s 的速度匀速下沉。运动轨迹表示为：

$$\mathbf{c} = \mathbf{s}_1^T - (0, 0, 3(t-5.1))^T \quad (5)$$

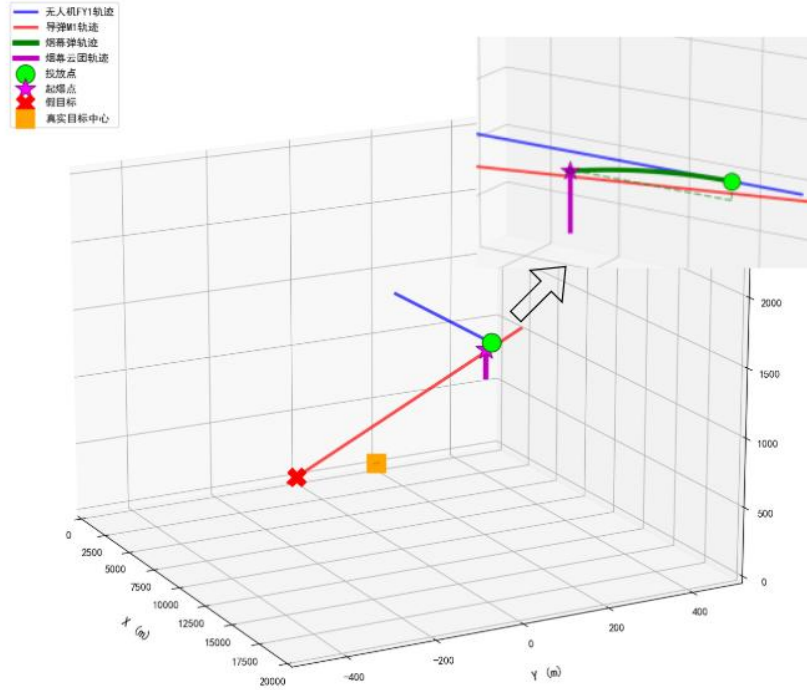


图 2 运动轨迹图

5.1.2 建立导弹-真实目标视线解析几何模型

判断是否遮蔽时，需要确定导弹到真实目标的视线是否能完全被云团遮挡，因此需要得到导弹到真实目标的视线方程。其中，真实目标圆柱体表面点 $p(x,y,z)$ 所满足的方程为：

$$x^2 + (y - 200)^2 = 7^2 (z \in [0,10]) \quad (6)$$

导弹与真实目标的视线即为导弹 M1 与圆柱表面点 p 构成的视线连线，将其表示为 l ，表达式为：

$$l = m - p \quad (7)$$

5.1.3 有效遮蔽判定条件

为了避免来袭导弹发现真目标，烟雾弹的投放应能使云团完全遮蔽真实目标的全貌，即导弹到真是目标的视线能完全被云团遮挡，因此通过计算云团中心 c 到所有视线的最大距离 $d_{\max}$ ，找到不大于 10m 的时间即为有效遮蔽时间。

$$d(t) = \frac{l \times (c - m)}{\|l\|} \quad (8)$$

但是因为计算云团中心到所有视线的距离会导致时间复杂度大大增加，因此综合考虑效率与精度，对真实目标圆柱上的点进行合理的采样，得到最优化的视线线段。

5.1.4 采样方法说明

本文选取的采样点共八个。从宽窄考虑，取真目标的上下表面与导弹的切点，上下两个共四个点，这四个切点就是导弹视角宽度的极限；从远近考虑，取面 MAB 即图 3 中虚线构成的平面，其中 A 为真目标上底面圆心，B 为真目标下底面圆心。切割

真目标上下底面的四个点（以下简称割点），这四个割点可以反应出导弹视角远近的极限。

求切点函数 $P(t, lis)$ ：根据“切点-对应圆心”连线垂直于“M-切点”和“M(z=0)-切点”特性，采用正交的方法直接求解。

求割点函数 $Q(t, lis)$ ：根据立体几何可以确定这四个割点坐标与 M 点坐标的关系：

$$Q_1(r \sin \theta, 200 - r \cos \theta, 0)、Q_2(-r \sin \theta, 200 + r \cos \theta, 0)$$

$$Q_3(r \sin \theta, 200 - r \cos \theta, 10)、Q_4(-r \sin \theta, 200 + r \cos \theta, 10)$$

其中 θ 是导弹 M 关于 xoy 平面（求 Q_3Q_4 时是平面 $Z=10$ ）的投影点（在 MAB 面上）与圆心的连线与 X 轴的夹角：

$$\sin \theta = \frac{M1x}{\sqrt{M1x^2 + 200^2}}$$

$$\cos \theta = \frac{200}{\sqrt{M1x^2 + 200^2}}$$

M1x 是导弹 M1 的 x 坐标值

表 2 示例时间点对应的 P 采样点坐标信息

T(s)	P1x	P1y	P1z	P2x	P2y	P2z	P3x	P3y	P3z	P4x	P4y	P4z
5	0.078	206.9	0	-0.072	193.0	0	0.078	206.9	10	-0.073	193.0	10
10	0.085	206.9	0	-0.079	193.0	0	0.085	206.9	10	-0.079	193.0	10

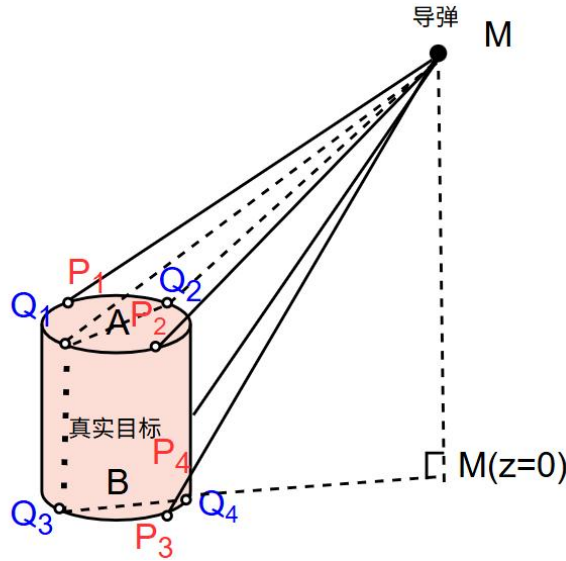


图 3 导弹采样点选取方式示意图

5.1.5 有效遮蔽时间计算

有效遮蔽时间为云团能完全把导弹到圆柱面上的所有视线全部遮挡的时间。这里为了平衡效率与计算精度，对圆柱上的点进行合理采样后，将导弹与采样点连线作为采样视线，若某时刻云团中心到这些视线的最远距离不超过 10m 时即为有效遮蔽时刻，公式表示为：

$$\begin{cases} d_{\max}(t) \leq 10 \\ t - t_s \leq 20 \end{cases} \quad (9)$$

根据根据此公式画出云团中心到视线最远距离 d 随时间的变化曲线，根据图线可

以看出云团的有效遮蔽的时间段为：

$$t \in [t_{\min}, t_{\max}] \quad (10)$$

结合图 4 则有效遮蔽时间为差值：

$$T_e = t_{\max} - t_{\min} \quad (11)$$

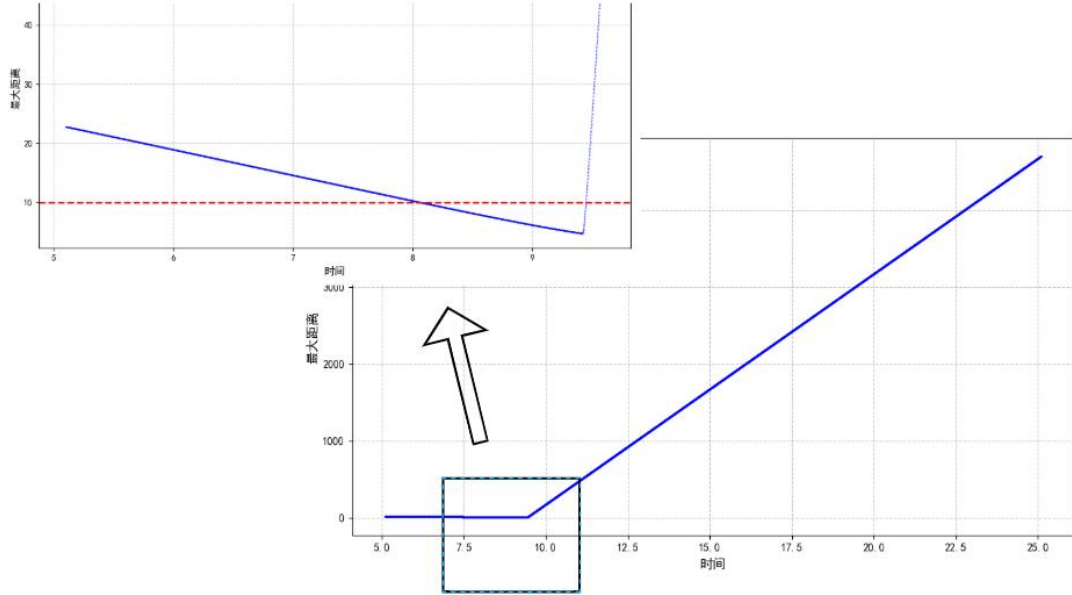


图 4 云团中心到视线最远距离 d 随时间的变化曲线

5.1.6 问题一的求解

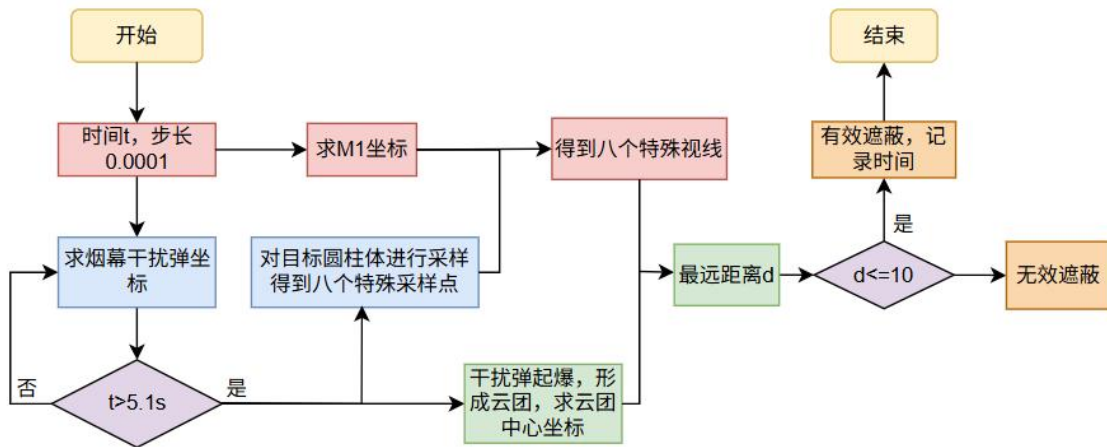


图 5 有效遮蔽时长求解步骤

要求解出总共的有效遮蔽时间。第一步，根据导弹运动轨迹模型和时间 t 求得此时得 m 点坐标；第二步，当 t 小于 5.1 秒时，使用无人机运动轨迹模型、烟幕弹运动轨迹模型和时间 t 求得此时烟幕弹位置，当 t 大于等于 5.1 秒时，使用云团运动轨迹模型和时间 t ，求解出云团中心位置；第三步，使用导弹-真实目标视线解析几何模型、和有效遮蔽判定条件进行最远距离判定，通过最远距离判定可以得出当前时间点，云团是否形成有效遮蔽，若形成有效遮蔽就记录当前时间点；第四步，输出有效遮蔽时

间，遍历完从导弹发射到云团消失的时间之后可以得知遮蔽的时间段，即可求出最大有效遮蔽时间。

从图 6 中可以得到最终结果为 1.3901s，有效遮蔽区间为 8.0563s~9.4464s。

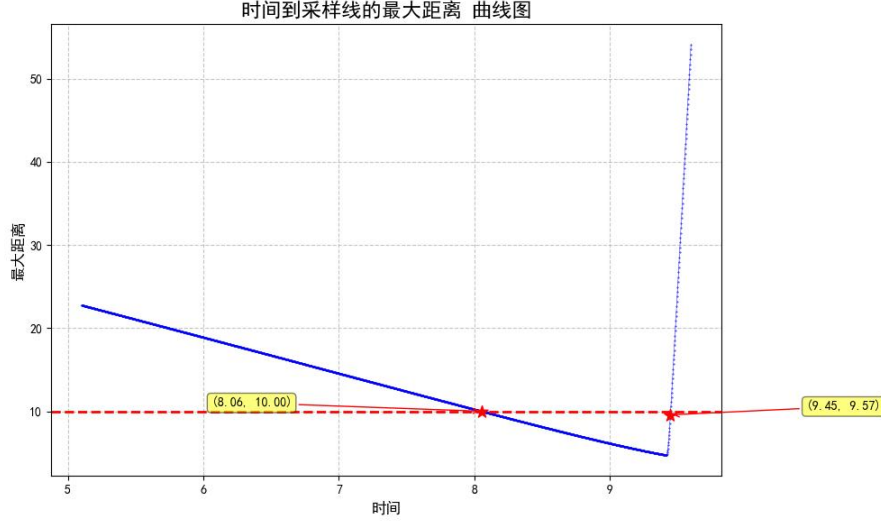


图 6 云团中心到视线最远距离 d 随时间的变化曲线

5.2 问题二的模型建立与求解

5.2.1 决策变量的确定

本文决策变量表示为：

$$x = [v_f, \alpha, t_s, \delta] \quad (12)$$

其中， v_f 为无人机 FY1 的速度标量， α 表示无人机方向角， t_s 表示烟幕弹投放时间， δ 表示烟幕弹起爆延迟时间。

5.2.2 约束条件的确定

速度大小约束：

无人机受领任务后，可根据需要瞬时调整飞行方向，然后以 70~140 m/s 的速度等高度匀速直线飞行：

$$v_f \in [70, 140] \quad (13)$$

烟幕弹需在落地前爆炸，否则造成烟幕弹浪费：

$$\frac{1}{2}g(t - t_s)^2 < 1800 \quad (14)$$

时间约束，时间为非负数：

$$t_s > 0 \quad (15)$$

起爆点在投放时间后起爆：

$$\delta \geq 0 \quad (16)$$

起爆点在导弹抵达假目标前爆炸：

$$t_s + \delta < 67 \quad (17)$$

5.2.3 三维轨迹模型的建立

导弹运行轨迹

$$\begin{aligned} \mathbf{m} &= \mathbf{m}_0 + \mathbf{v}_{m1} \times \mathbf{n} \cdot t \\ \mathbf{m}_0 &= (20000, 0, 2000) \end{aligned} \quad (18)$$

无人机运行轨迹

根据决策变量 $\mathbf{v}$, 无人机 x 、 y 轴上分别由速度分量, 则无人机运行轨迹可表示为:

$$\mathbf{f} = \mathbf{f}_0 + (\mathbf{v}_f \cos \alpha, \mathbf{v}_f \sin \alpha, 0)^T t \quad (19)$$

当 $t=t_s$ 时, 无人机此时坐标为 $\mathbf{f}_1$, 烟雾干扰弹投放点即在此处

烟雾干扰弹运行轨迹

烟雾干扰弹在投放后与起爆前的水平方向速度与无人机相同, 竖直方向受重力作用, 做评判运动:

$$\mathbf{s} = \mathbf{f}_1 + \left(\mathbf{v}_f \cos \alpha, \mathbf{v}_f \sin \alpha, \frac{1}{2} g t \right)^T t \quad (20)$$

当 $t=t_s + \delta$ 时, 烟雾干扰弹起爆, 位置为 $\mathbf{s}_1$

烟雾云团运行轨迹

烟雾弹爆炸后瞬时形成初始球状云团, 云团的中心位置假设为烟雾干扰弹起爆位置即 $\mathbf{s}_1$, 此后, 云团水平方向速度为零, 竖直方向以 3m/s 的速度匀速下沉。运动轨迹表示为:

$$\mathbf{c} = \mathbf{s}_1 - (0, 0, 3)^T (t - t_2) \quad (21)$$

5.2.4 目标函数

在无人机 FY1 烟雾弹投放策略优化中, 核心目标是最大化烟雾对真实目标的有效遮蔽时间 T_e , 表达为:

$$T_e = \int_0^{t_{\max}} I(t) dt \quad (22)$$

$$I(t) = \begin{cases} 1, & d(t) \leq 10 \\ 0, & d(t) > 10 \end{cases} \quad (23)$$

数值近似采用时间离散化, 步长 $\Delta t = 0.01 \text{ s}$:

$$T_e \approx \sum_{k=0}^K I(t_k) \cdot \Delta t, t_k = k \cdot \Delta t \quad (24)$$

5.2.5 模型汇总

$$\begin{aligned} \max T_e &= \int_0^t I(t) dt \\ I(t) &= \begin{cases} 1, & d(t) \leq 10 \\ 0, & d(t) > 10 \end{cases} \\ s.t. &\begin{cases} v_f \in [70, 140] \\ \frac{1}{2} g(t-t_s)^2 < 1800 \\ t_s > 0 \\ \delta \geq 0 \\ t_s + \delta < 67 \end{cases} \end{aligned} \quad (25)$$

5.2.6 基于遗传算法的无人机 FY1 烟幕弹投放策略优化

遗传算法是一种逐步进化到搜索空间的更优区域的随机全局搜索优化方法。

本问的核心目标是最大化烟幕对真实目标的有效遮蔽时间 T_e ，优化参数包括无人机速度 v_f 、无人机方向角 θ 、投放时间 t_d 、起爆延迟 $\delta = t_s - t_d$ ，且需满足速度范围、起爆点高度非负等物理约束。传统局部优化算法易陷入局部最优，而遗传算法通过维持种群多样性、实现基因交换，能在更大范围内搜索全局最优解，并在编码、交叉、变异操作上都进行了改进。更适配本题“多约束、高维、非线性”的复杂优化需求。

Step1: 个体编码与种群表示

在编码策略上，摒弃传统二进制编码，采用实数编码直接对应四维决策变量。遗传算法中的“个体”对应一组完整的决策变量向量 x ，种群则是个体的集合 $P = \{x_1, x_2, \dots, x_N\}$ ，其中 N 为种群规模，本题设 $N = 50$ 。在编码策略上，摒弃传统二进制编码，采用实数编码直接对应四维决策变量

个体编码：采用实数编码，每个个体的基因向量为：

$$x_i = [v_f, i, \alpha_i, t_{d,i}, \delta_i]^T, \quad i = 1, 2, 3, \dots, 50 \quad (26)$$

初始种群生成：为了保证初始多样性，每个个体的各分量需在约束范围内随机均匀生成，即对第 i 个个体的第 j 个分量 ($j = 1, 2, 3, 4$ 对应上述 4 个参数)：

$$x_{i,j} \sim Uniform(lower_j, upper_j) \quad (27)$$

其中 $lower_j$ 、 $upper_j$ 分别为第 j 个变量的下界与上界。

Step2: 适应度函数

适应度函数是评估个体“优劣”的核心指标，和优化目标（有效遮蔽时间）直接相关，其定义如下：

$$f(x) = \begin{cases} 1, T_e(x) & \text{若 } x \text{ 满足所有物理约束} \\ 0, Viol(x) & \text{若 } x \text{ 违反约束} \end{cases} \quad (28)$$

其中： $T_e(x)$ 是可行解的有效遮蔽时间（非负，值越大表示个体越优）， $Viol(x)$

表示约束违反。针对本问，寻找使适应度最大的决策变量向量是优化目标，即：

$$x^* = \arg \max_{x \in X} f(x) \quad (29)$$

其中 X 为所有满足物理约束的决策变量集合。

Step3: 选择操作

采用基于适应度的概率选择锦标赛选择算法基于适应度的概率选择，保留高适应度个体，进而引导种群进化方向。

Step4: 交叉操作

采用混合交叉策略，通过扩展父代变量区间生成子代，增加基因多样性。具体公式如下：

$$\begin{cases} L'_j = L_j - \alpha \cdot (U_j - L_j) \\ U'_j = U_j + \alpha \cdot (U_j - L_j) \\ x_{c,j} = L'_j + r_j \cdot (U'_j - L'_j) \end{cases} \quad (30)$$

每组父代生成 2 个子代 x_{c1}, x_{c2} 加入子代池，最终子代池规模与父代池一致 ($N = 50$)，确保种群规模稳定。

Step5: 变异操作

首先进行变异概率判定，对子网代池中的每个个体 x 、每个分量 j 生成随机数 $r \sim \text{Uniform}(0,1)$ ，若 $r \leq p_m = 0.2$ 随机数满足条件（变异概率设为 0.2），则执行变异操作，否则保留原分量；接着进行变异计算，变异后的分量

$$x'_j = x_j + N(0, \sigma_j^2) \quad (31)$$

式中， $N(0, \sigma_j^2)$ 与均值为 0、方差为 σ_j^2 的高斯随机变量。

Step6: 迭代与收敛

遗传算法通过“多代迭代”逐步逼近最优解，且需明确迭代终止条件与结果输出规则。首先，初始化一个种群大小为 N 的种群，设置最大迭代次数 $G_{\max} = 100$ 和收敛阈值 $\epsilon = 10^{-3}$ ，初始化当前迭代次数 $g = 0$ ，初始种群 P_0 （按 Step1 生成），计算 P_0 中所有个体的适应度，记录初始最优适应度 $f_{\text{best}}(g = 0) = \max \{f(x) | x \in P_0\}$ ，然后进入迭代循环，当 $g < G_{\max}$ 且未满足收敛条件时，重复执行选择操作、交叉操作、变异操作、适应度更新，同时每次迭代后进行收敛判定，若 $|f_{\text{best}}(g + 1) - f_{\text{best}}(g)| \leq \epsilon$ （连续两代最优适应度变化小于阈值），且该状态持续 $K = 5$ 代（避免偶然波动误判收敛），则判定算法收敛，提前停止迭代。

迭代停止后，输出所有迭代过程中的全局最优个体：

$$x^* = \arg \max \{f_{\text{best}}(g) | g = 0, 1, \dots, G_{\text{final}}\} \quad (32)$$

其中 G_{final} 为实际终止迭代次数， x^* 对应的 $[v_F^*, \alpha^*, t_d^*, \delta^*]$ 即为无人机 FY1 烟幕弹投放的最优策略。

算法流程图如下：

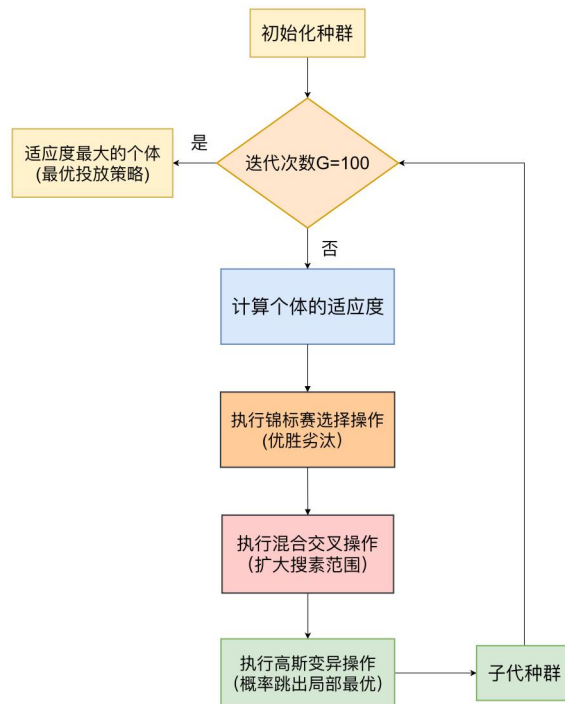


图 7 遗传算法迭代流程

5.2.7 模型求解结果

从决策变量的优化过程图中可以看出，随着迭代次数的增加，决策变量也逐渐收敛最优。下面是该过程中适应度，即有效遮蔽时间的变换情况。

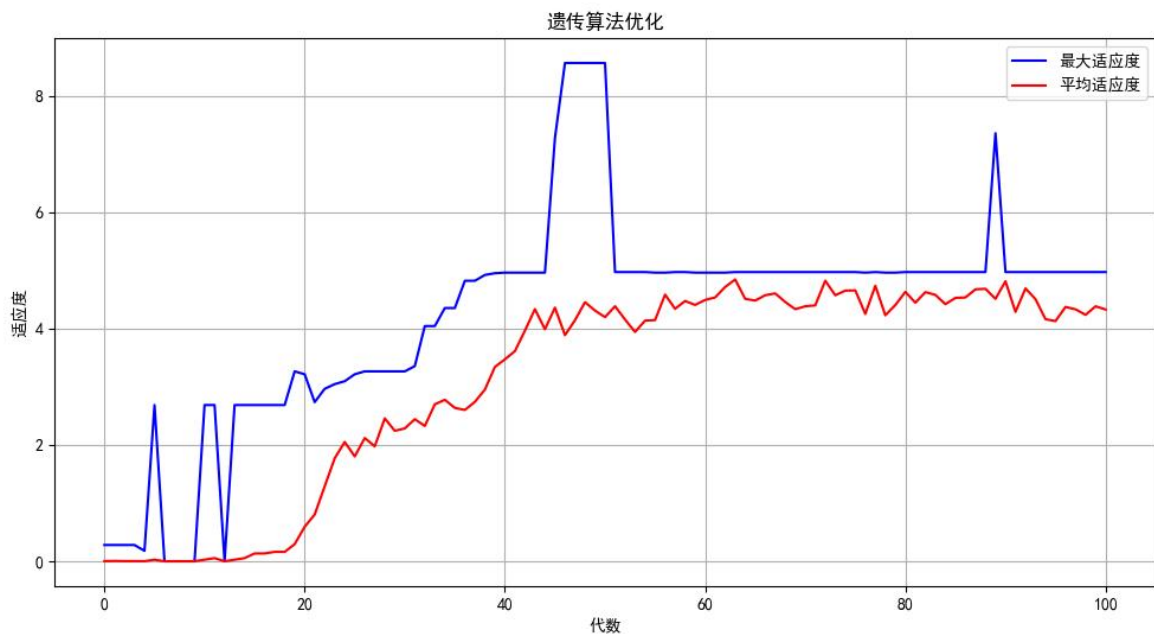


图 8 适应度函数值随迭代次数的变化曲线

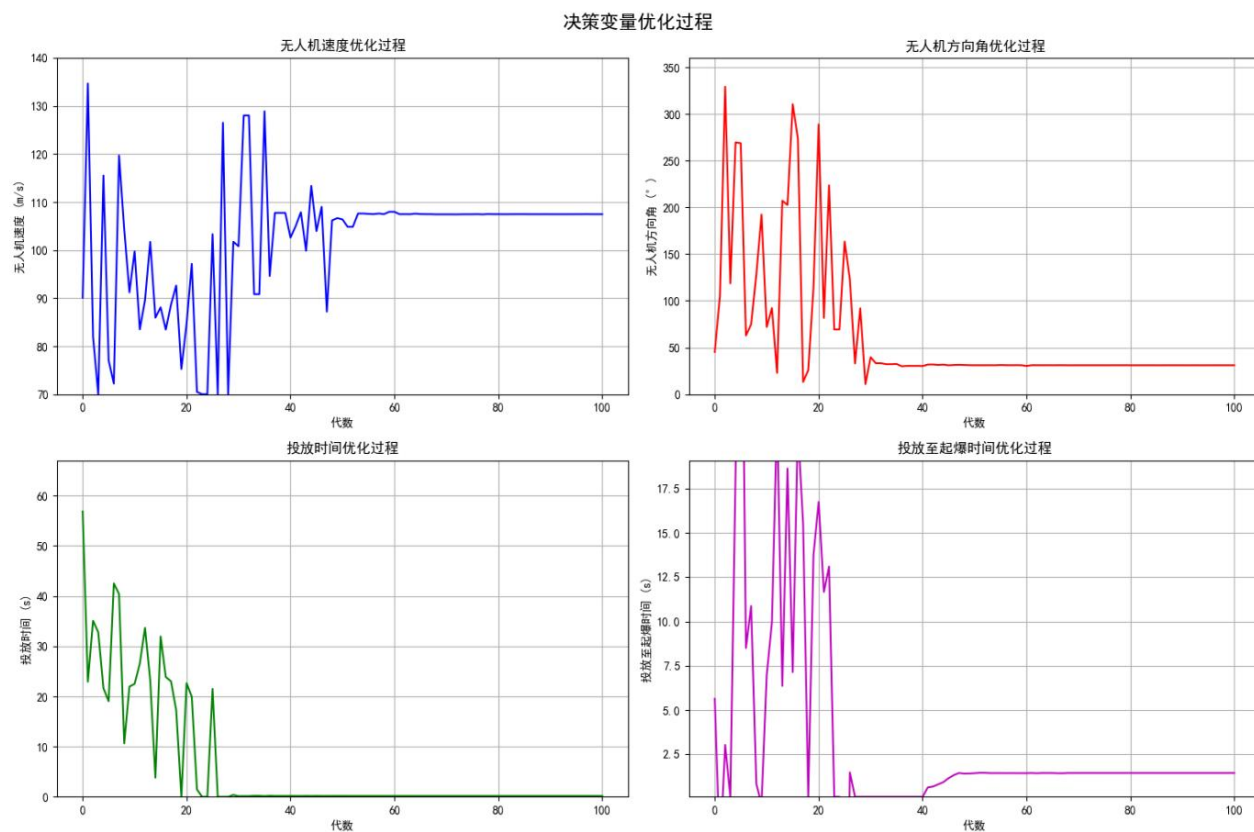


图 9 决策变量随迭代次数的变化曲线

可以明显看出，随着迭代次数的增加，有效遮蔽时间在总体趋势上是上升的，且最终收敛到一个最大值。下面是烟幕云团的遮蔽效果时间序列：

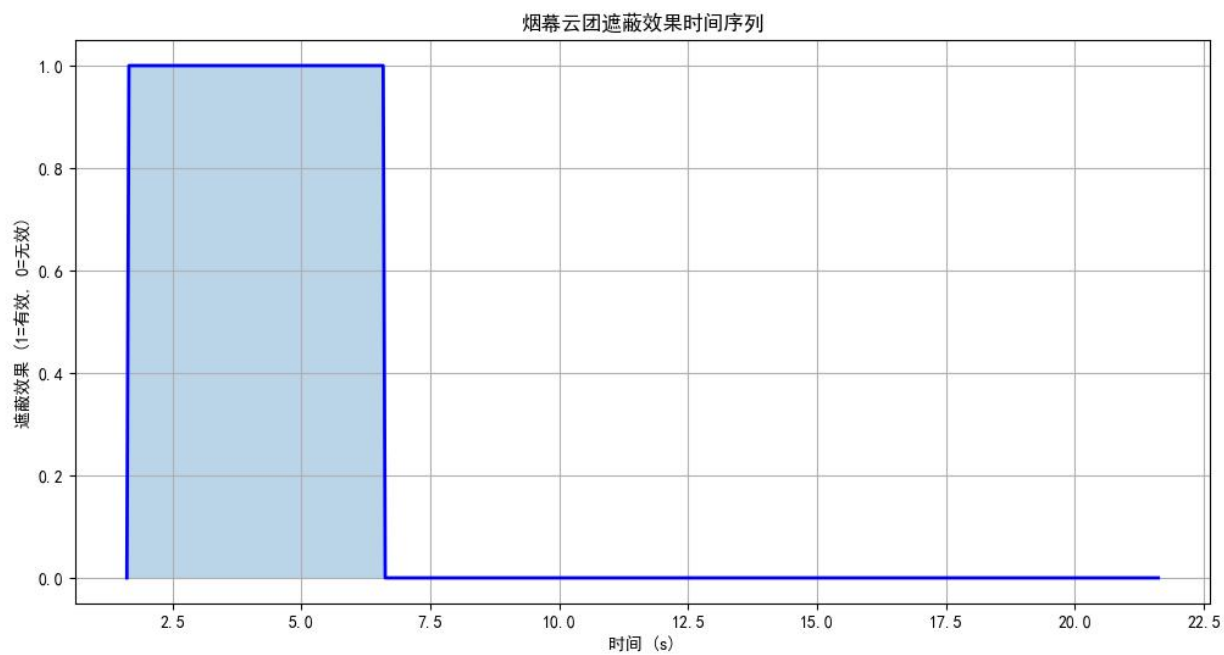


图 10 烟幕云团遮蔽时间序列

最终的优化结果为:

最大有效遮蔽时间: 4.96 s

无人机飞行速度: 102.13 m/s

无人机飞行方向角: 144.84°

烟幕干扰弹投放时间: 0.16 s

烟幕干扰弹投放点坐标: (17786.43, 9.55, 1800.00)

烟幕干扰弹起爆时间: 1.81 s

烟幕干扰弹起爆点坐标: (17786.43, 9.55, 1786.65)

5.3 问题三模型的建立与求解

5.3.1 决策变量

设置了一个八维决策变量:

$$\mathbf{x} = [\alpha, v_f, t_{s,1}, \delta_1, \beta_{s,2}, \delta_2, \beta_{s,3}, \delta_3] \quad (33)$$

其中, α 为无人机 FY1 的飞行方向, v_f 为无人机的速度标量, δ_i 为延迟爆炸时间, $\beta_{s,i}$ 为第 1 次与第 2 次、第 2 次与第 3 次投放烟幕干扰弹的时间间隔。

5.3.2 约束条件

时间顺序约束, 所有烟幕干扰弹的投放不得早于起爆时间:

$$t_{s,i} \leq t_{s,i} + \delta_i \quad (34)$$

题目中要求每架无人机投放两枚烟幕干扰弹至少间隔 1 s, 表达如下:

$$\beta_{s,i} \geq 1 (i = 2, 3) \quad (35)$$

题目中要求无人机受领任务后以 70~140 m/s 的速度等高度匀速直线飞行。无人机速度约束:

$$70 \leq v_f \leq 140 \quad (36)$$

题目中设定云团中心 10 m 范围内的烟幕浓度在起爆 20s 内可为目标提供有效遮蔽, 因此云团有效遮蔽时间约束为:

$$t_{s,i} + \delta_i \leq t \leq t_{s,i} + \delta_i + 20 \quad (37)$$

为了不浪费任何一枚干扰弹, 每一个干扰弹在落地之前都应能爆炸:

$$\frac{1}{2} g \delta^2 \leq 1800 \quad (38)$$

5.3.3 目标函数

目标找到有效遮蔽时间最长的策略, 设时间离散化步长为 dt , 则建立关于 t 的指示函数:

$$I(t) = \begin{cases} 1, & d(t)_{\max} \leq 10 \\ 0, & d(t)_{\max} > 10 \end{cases} \quad (39)$$

$$T_e = \sum I(t) \cdot dt \quad (40)$$

为使得遮蔽时间尽可能长，此目标函数为：

$$\max T_e \quad (41)$$

因为使用差分进化优化算法进行优化时默认为最小化目标函数，因此将目标函数改写为：

$$\min M = -T_e \quad (42)$$

5.3.4 模型汇总

$$\begin{aligned} \min M = -T_e \\ \left\{ \begin{array}{l} t_{s,i} \leq t_{s,i} + \delta_i \\ \beta_{s,i} \geq 1 (i = 2,3) \\ 70 \leq v_f \leq 140 \\ t_{s,i} + \delta_i \leq t \leq t_{s,i} + \delta_i + 20 \\ \frac{1}{2} g \delta^2 \leq 1800 \end{array} \right. \end{aligned} \quad (43)$$

5.3.5 基于差分进化的投放策略优化模型求解

1. 初始化：定义时间序列步长为 0.01，总时长为 67s。在圆柱上下圆周上进行采样，建立完整的几何遮蔽判定体系。

2. 适应度函数计算：采用创新的增量参数表示法，计算 3 枚烟幕弹的投放点、起爆点及烟幕云轨迹。逐时刻判定是否所有关键点被遮蔽，采用"任一云团遮蔽关键点即可"的联合判定准则，合并连续有效区间并计算总遮蔽时间。返回总遮蔽时间的负值。

3. 差分进化优化：迭代搜索最优参数。

种群初始化：设置种群规模为 80，在智能边界约束范围内随机初始化。

变异操作：采用 best1bin 策略，以当前最优个体引导搜索方向，平衡全局探索与局部开发。

交叉操作：设置重组概率为 0.8，促进优良基因传播。

选择操作：采用贪婪选择策略，保留更优个体。

参数调优：经过 40 代迭代，结合适应度函数值确保获得高质量解。

约束处理：通过边界约束直接保证投放间隔要求，避免惩罚函数复杂度。

4. 结果分析与输出：计算每枚烟幕弹的单独遮蔽时间、投放点 / 起爆点坐标，可视化轨迹。观察图 11，不断迭代最后到 6.17s。

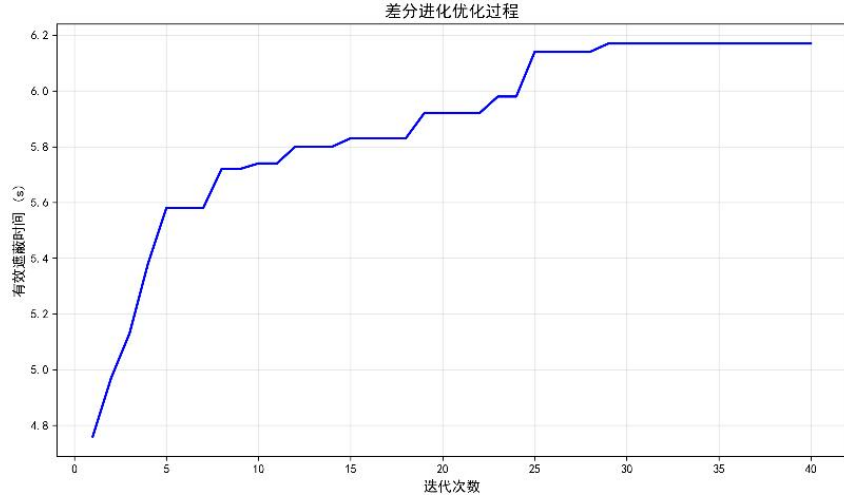


图 11 差分优化过程

5.4 问题四的模型建立与求解

本问实际上是在问题 2 的基础上进行了拓展，增加了无人机的数量，从 1 架无人机投放 1 枚烟幕弹拓展至 3 架无人机各投放 1 枚烟幕弹，要求设计烟幕弹投放策略，以最大化烟幕对来袭导弹 M_1 遮蔽真目标的有效时间。

分析题目可知，与问题 2 相比，问题 4 并未增加约束条件，仅仅是增加了决策变量的数量。因此，我们可以沿用问题 2 的解题思路，利用遗传算法，针对种群的每个个体修改其编码以增加决策向量进行求解。最终的优化结果，即为 3 架无人机情况下的最优烟幕弹投放策略。

针对本题，总的目标函数不变：

$$T_e = \int_0^{t_{\max}} I(t) dt \quad (44)$$

数值近似采用时间离散化（步长 $\Delta t = 0.01 s$ ）：

$$T_e \approx \sum_{k=0}^K I(t_k) \cdot \Delta t, t_k = k \cdot \Delta t \quad (45)$$

对于遗传算法中目的每个个体的编码进行修正：

$$x = (v_F^{(1)}, \theta^{(1)}, t_d^{(1)}, t_s^{(1)}, \dots, v_F^{(M)}, \theta^{(M)}, t_d^{(M)}, t_s^{(M)}) \quad (46)$$

其中 $\mathbf{x}$ 为个体为实数向量， M 为无人机数量（ $M = 3$ ）。

模型汇总

$$\begin{aligned}
\max T_e &= \int_0^t I(t) dt \\
I(t) &= \begin{cases} 1, d(t) \leq 10 \\ 0, d(t) > 10 \end{cases} \\
\text{st.} &\begin{cases} \|v\| \in [70, 140] \\ \frac{1}{2}gt^2 < 1800 \\ t > 0 \\ t_d > 0 \\ t_s > t_d \\ t_s < 67 \end{cases}
\end{aligned} \tag{47}$$

依然采用遗传算法进行求解，求解结果如下：

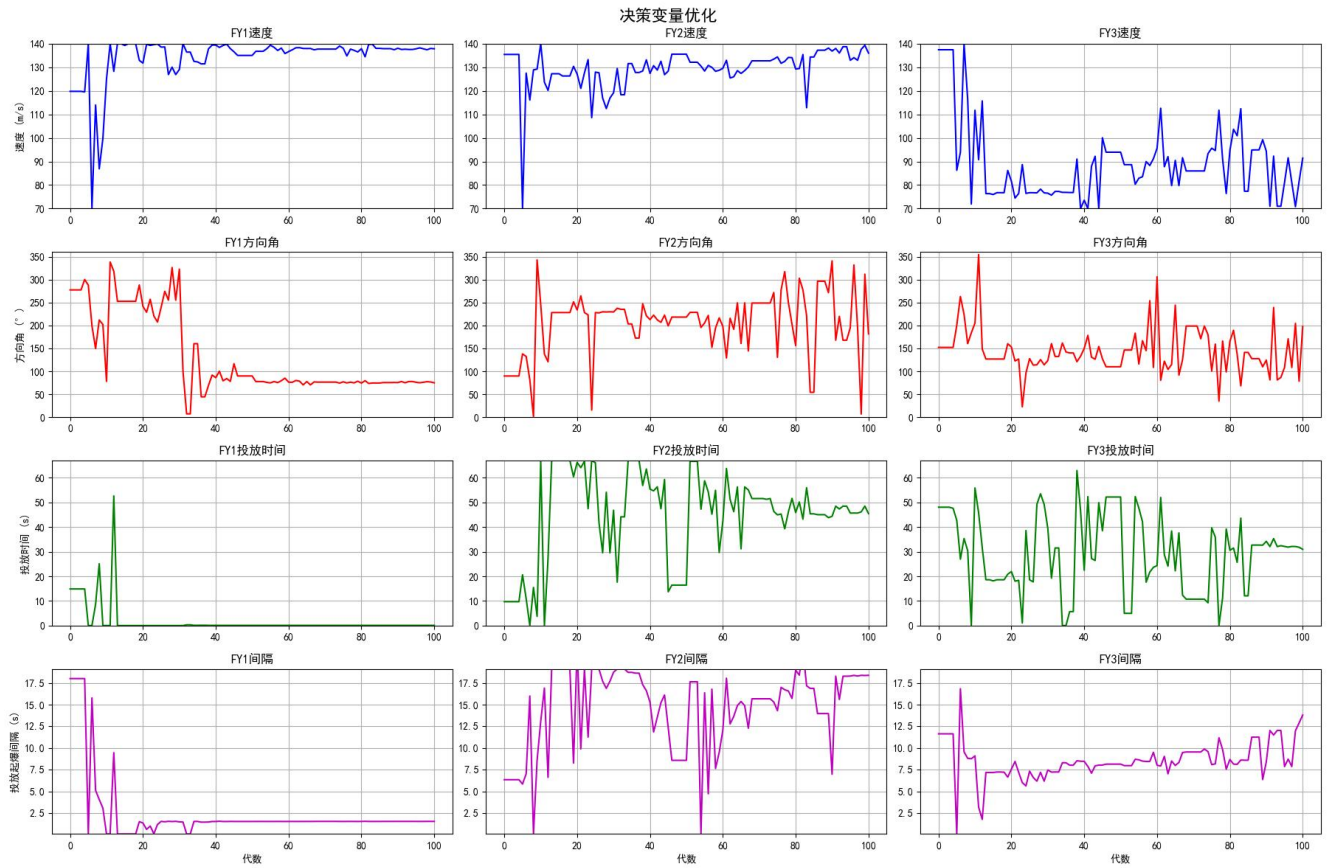


图 12 各个决策变量随迭代次数的变化

阿斯顿从图中可以看出，随着迭代次数的增加，决策变量也逐渐收敛最优。下面是该过程中适应度，即有效遮蔽时间的变化情况。

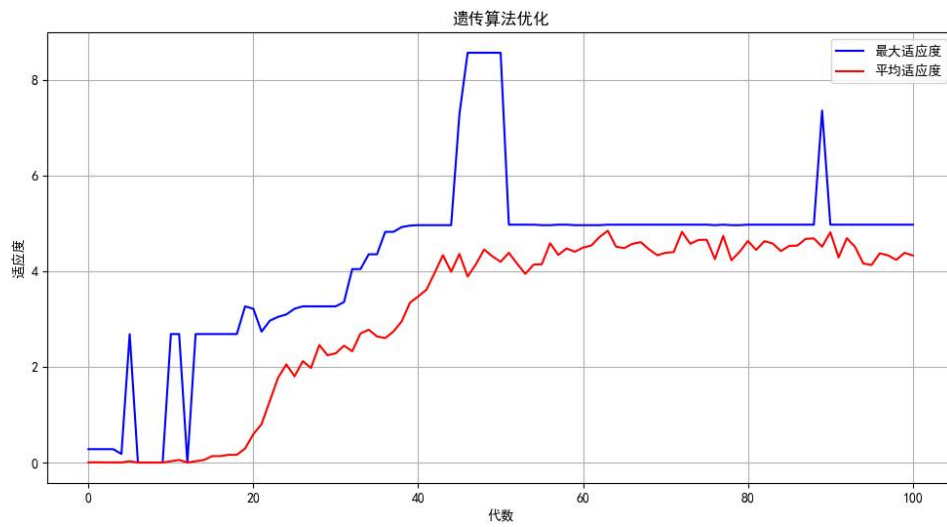


图 13 适应度随迭代次数变化情况

可以明显看出，随着迭代次数的增加，有效遮蔽时间在总体趋势上是上升的，且最终收敛到一个最大值。

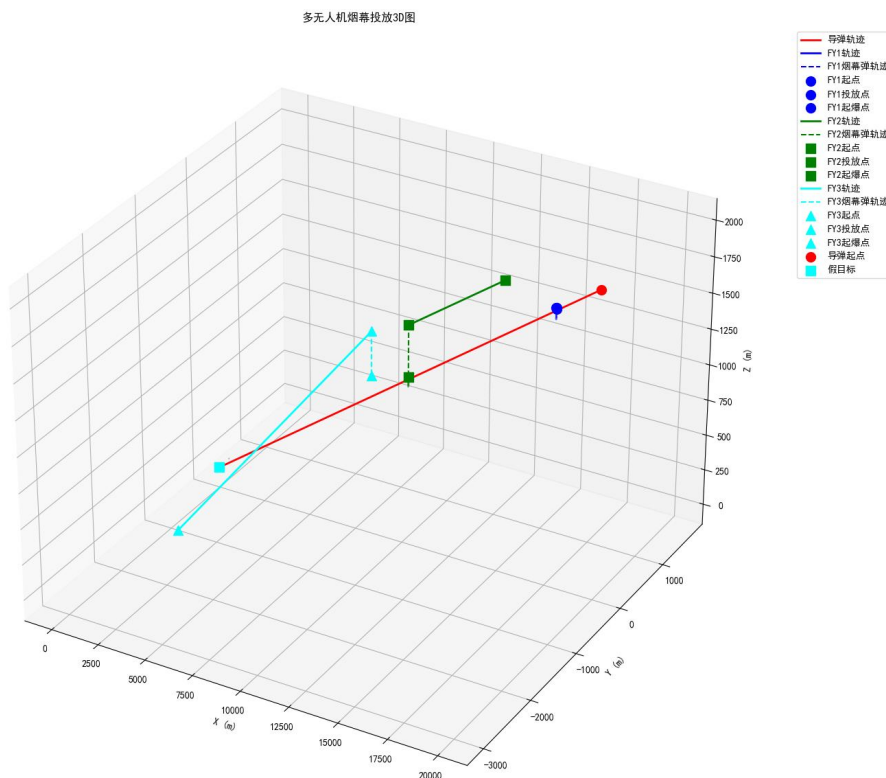


图 14 无人机投放策略图

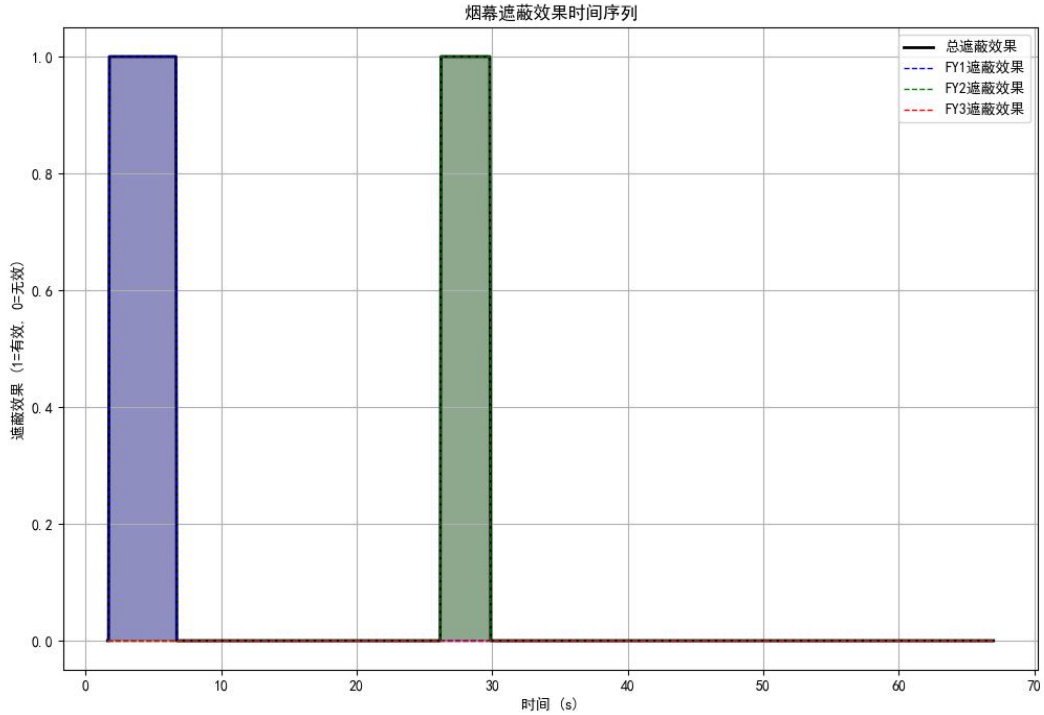


图 15 烟雾遮蔽情况随时间分布图
最终有效遮蔽时间为 8.58s。优化后的最终策略如表 3 所示。

表 3 最终优化结果

无人机编号	运动方向	运动速度 (m/s)	投放点 x (m)	投放点 y (m)	投放点 z (m)	起爆点 x (m)	起爆点 y (m)	起爆点 z (m)	有效时间 (s)
FY1	89.72	135.00	17800.05	9.57	1800.00	17800.05	9.57	1788.36	4.96
FY2	218.14	135.52	10250.07	25.76	1400.00	10250.07	25.76	1039.07	3.62
FY3	109.80	93.95	4339.13	1613.18	700.00	4339.13	1613.18	374.92	0.00

5.5 问题五的模型建立与求解

针对问题 5 这种多无人机-多导弹-多烟雾弹的协同优化问题，如果单纯使用遗传算法求解的话会因求解维度过多而陷入维度灾难。因此需在问题 4 的基础上，制定无人机-导弹的针对干扰策略，通过合理分配无人机拦截对应导弹，来缩小解空间的范围。本题考虑使用贪心算法制定无人机分配策略，以及每个无人机的投弹数量分配策略，然后根据贪心算法生成的初始解，利用遗传算法进行进一步的优化求解。

首先，设无人机集合： $U = \{U_1, U_2, U_3, U_4, U_5\}$ 导弹集合： $M = \{M_1, M_2, M_3\}$ ，设弹药集合 $K = K_1, K_2, \dots, K_m$ ，初始坐标均已知，弹药总数未知。

同时，设指示变量 $D_{i,j}=1$ 表示第 i 架无人机 U_i 专门干扰第 j 枚导弹 M_j ，否则值为 0，表示不干扰。

新增约束 1: $\sum_j D_{i,j} \leq 1 \ (\forall i)$ ——“每架无人机至多干扰 1 枚导弹”

新增约束 2: $\sum_i D_{i,j} \geq 1 \ (\forall j)$ ——“每枚导弹至少被 1 架无人机干扰”

二、贪心算法求初始解的两阶段过程

贪心算法的核心逻辑是逐次选择局部最优决策，最终生成“无人机 - 导弹任务分配 + 弹药数量初分”的初始解，分两阶段执行：

阶段 1: 无人机 - 导弹任务分配

步骤 1.1 构建适配度函数 $F(i, j)$ (量化局部最优匹配)

适配度函数数学表达式为：

$$F(i, j) = \alpha \cdot F_1(i, j) + \beta \cdot F_2(i, j) + \gamma \cdot F_3(i, j), \text{ 其中 } \alpha + \beta + \gamma = 1$$

式中，距离适配项 $F_1(i, j)$: U_i 初始位置到 M_j 轨迹的距离越近，干扰响应越快。

时间匹配项 $F_2(i, j)$: U_i 投放烟幕的时间需覆盖 M_j 飞行窗口，

高度适配项 $F_3(i, j)$: U_i 初始高度需适配烟幕下沉，确保烟幕覆盖真目标

步骤 1.2 贪心选择确定 $D_{i,j}$

计算所有 $(i, j) \in \{1, \dots, 5\} \times \{1, 2, 3\}$ 的 $F(i, j)$ ，生成适配度矩阵 $\mathbf{F} = [F(i, j)]_{5 \times 3}$ ；

对 $\mathbf{F}$ 中所有元素按降序排序，得到优先级序列 $\mathcal{S} = \{(i_1, j_1), (i_2, j_2), \dots, (i_{15}, j_{15})\}$ ，满足 $F(i_1, j_1) \geq F(i_2, j_2) \geq \dots \geq F(i_{15}, j_{15})$ ；

初始化任务分配矩阵 $\mathbf{D} = [D_{i,j}]_{5 \times 3} = \mathbf{0}$ ，导弹覆盖标记 $\mathcal{C} = \{1, 2, 3\}$ ，迭代遍历 $\mathcal{S}$ ：

对当前 $(i, j) \in \mathcal{S}$ ，若 $\sum_{j'=1}^3 D_{i,j'} = 0$ 且 $j \in \mathcal{C}$ ，则令 $D_{i,j} = 1$ ，并从 $\mathcal{C}$ 中移除 j ；

若 $\mathcal{C} = \emptyset$ (所有导弹已覆盖)，终止迭代；

阶段 2: 弹药数量初分配确定 K_i

步骤 2.1 定义弹药覆盖时长函数 $T_e(i, k)$ 对已分配导弹 j 的 U_i ，投放 k 枚烟幕弹的最大可能遮蔽时长 $T_e(i, k)$

$$T_e(i, k) = \min \left\{ T_{S,e} + (k - 1) \cdot (T_{S,e} - 1), T_{M_j} - t_{U_i, \min} \right\}. \quad (48)$$

步骤 2.2 贪心选择确定 K_i

首先初始化弹药数量向量 $\mathbf{K} = [K_1, K_2, K_3, K_4, K_5]^T = \mathbf{0}$ ，剩余弹药数 $N_{rem} = 15$ ，空白遮蔽区间 $\Omega = [0, T_{\max}]$ ，然后进行迭代筛选，直至 $N_{rem} = 0$ 或 $\Omega = \emptyset$ 。

三、初始解的数学形式

通过上述两阶段贪心过程，最终生成的初始解为：

任务分配矩阵 $\mathbf{D}^0 = [D_{i,j}^0]_{5 \times 3}$ ，满足 $\sum_{j=1}^3 D_{i,j}^0 \leq 1 \ (\forall i)$ 、 $\sum_{i=1}^5 D_{i,j}^0 \geq 1 \ (\forall j)$ ；

弹药数量向量 $\mathbf{K}^0 = [K_1^0, K_2^0, K_3^0, K_4^0, K_5^0]^T$ ，满足 $0 \leq K_i^0 \leq 3 \ (\forall i)$ 、 $\sum_{i=1}^5 K_i^0 \leq 15$ ，

这样求出的初始解为后续遗传算法优化提供了合理的决策框架，避免了直接优化

高维变量的“维度灾难”。

遗传算法求最优解

根据贪心得出初始分配方案，将问题 5 转化为若干个问题 3 和问题 4，最后利用问题 3 和问题 4 的求解方法，最大化多无人机-多导弹-多烟幕弹系统的总有效遮蔽时间。

模型汇总：

$$\begin{aligned}
 \max T_e &= \int_0^t I(t) dt \\
 I(t) &= \begin{cases} 1, d(t) \leq 10 \\ 0, d(t) > 10 \end{cases} \\
 \text{st.} &\begin{cases} \|v\| \in [70, 140] \\ \frac{1}{2}gt^2 < 1800 \\ t > 0 \\ t_d > 0 \\ t_s > t_d \\ t_s < 67 \\ \sum_{i=1}^5 K_i \leq 15 \\ K_i \leq 3 \\ \sum_{j=1}^3 D_{i,j} \leq 1 \\ \sum_{i=1}^5 D_{i,j} \geq 1 \end{cases}
 \end{aligned} \tag{49}$$

模型求解结果

从各无人机对总有效遮蔽时间的贡献图可以看出，大部分无人机对总有效遮蔽时间都有贡献，而无人机 FY4 对总有效遮蔽时间的贡献为 0，猜测可能是由于改无人机分配的导弹还有另一架无人机进行干扰，且覆盖了无人机 FY4 的干扰，从而导致其贡献为 0。

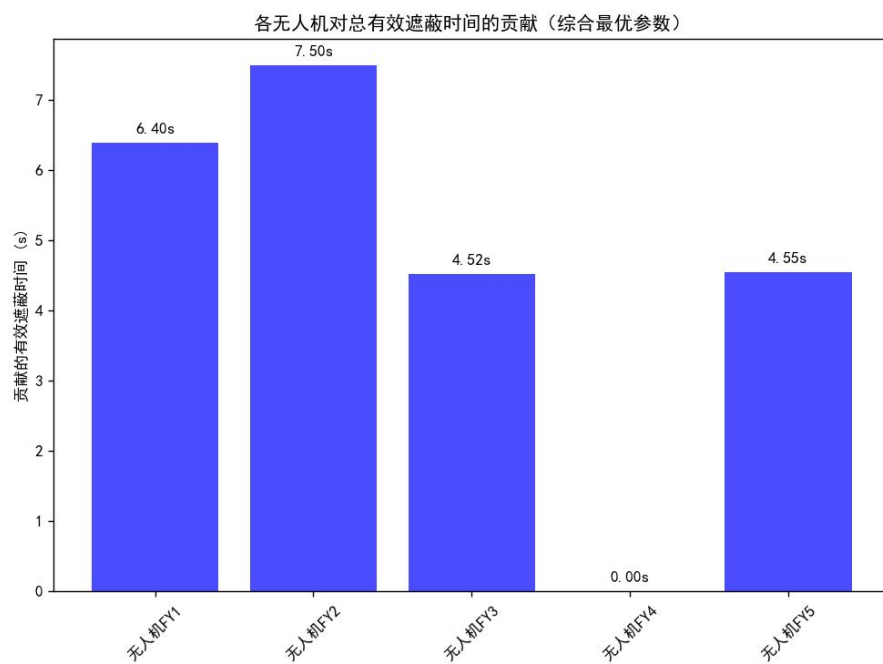


图 16 无人机对有效时间贡献
下面是三枚导弹的有效遮蔽时间分布图。

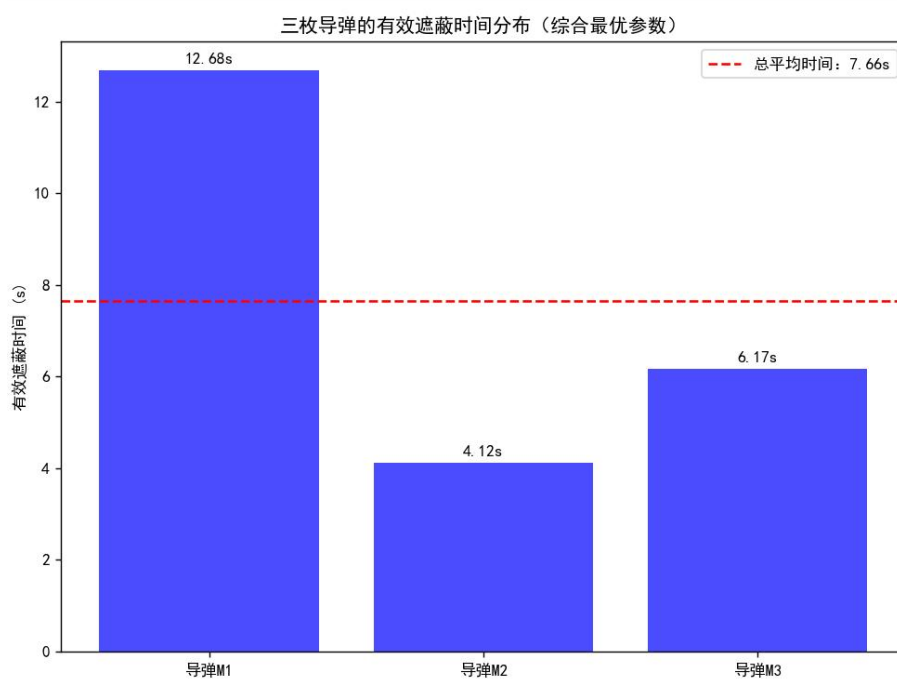
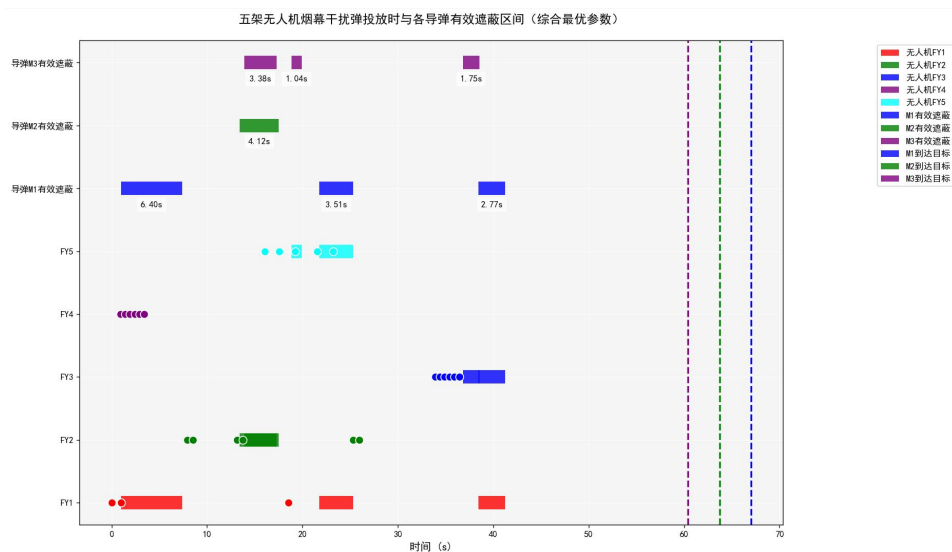


图 17 三枚导弹的有效遮蔽时间分布图
下面是对五架无人机烟幕干扰弹投放时与各导弹的有效遮蔽区间分析图。



图五架无人机烟幕干扰弹投放时与各导弹的有效遮蔽区间

六、 模型的评价、改进与推广

6.1 模型的优点

问题三采用差分进化算法进行参数优化，能够在合理时间内找到近似最优解，适用于“高维，多参数，多约束”情况中的快速决策需求。

所有优化都模型充分考虑了实际物理约束，如无人机速度范围、投放间隔限制、云团下沉特性等，结果具有较高的实用价值。

问题二所用的遗传算法优化模型能够应用在问题三解决问题，其处理单枚和多枚烟幕干扰弹的协同投放问题，可扩展性强。

提供了三维轨迹图和优化过程图，直观展示了烟幕遮蔽过程和算法收敛情况。

6.2 模型的缺点

模型计算复杂度较高对于多无人机多导弹的复杂场景，计算时间较长。

所有对象都假设无阻力，实际生活应用受限

6.3 模型的改进

针对多无人机多导弹场景，可采用并行计算技术提高求解效率，满足实战中的实时性要求。

引入风速、风向、大气稳定性等环境参数，建立更精确的烟幕扩散模型。

6.4 模型的推广

本模型框架可推广至红外干扰弹、箔条干扰弹等其他类型干扰弹的投放策略设计。可进一步考虑无人机与地面发射装置、有人机等其他平台的协同作战，形成立体防护体系。

七、参考文献

- [1]徐路程,郝雪颖,肖凯涛,等.爆炸型烟幕弹遮蔽效能仿真研究[J].兵工学报,2020,41(07):1299-1306.
- [2]蒋硕,袁小平.改进 PSO 算法在多无人机协同任务分配中的应用[J].计算机应用研究,2019,36(11):3344-3347+3360.DOI:10.19734/j.issn.1001-3695.2018.05.0303.
- [3]张耀中,陈岚,史国庆,等.时序耦合约束下的多无人机协同任务决策研究[J].西北工业大学学报,2018,36(05):890-896.
- [4]郭爱强,李天鹏,李笑楠,等.烟幕干扰激光末制导炮弹的仿真研究（英文）[J].红外与激光工程,2022,51(04):284-294.
- [5]丁佳麟,陈春生,李庆伟,等.烟幕干扰效果评估指标及计算方法[J].红外,2024,45(03):29-39.
- [6] DeepSeek, DeepSeek-R1-0528, 深度求索（DeepSeek）, 2025-09-05
- [7] Doubao, Doubao-Seed-1.6, 字节跳动, 2025-09-05

附录

附录 1 支撑材料文件列表

文件名	说明
p1.py	问题 1 中计算烟幕干扰弹对 M1 的有效遮蔽时长
p2.py	问题 2 中获取最有烟幕弹投放策略和有效遮蔽时长
p3.py	问题 3 中获取最有烟幕弹投放策略和有效遮蔽时长
p4.py	问题 4 中获取最有烟幕弹投放策略和有效遮蔽时长
p5.py	问题 5 中获取最有烟幕弹投放策略和有效遮蔽时长
result1.xlsx	保存问题 3 中有关烟幕弹投放策略的参数信息
result2.xlsx	保存问题 4 中有关烟幕弹投放策略的参数信息
result3.xlsx	保存问题 5 中有关烟幕弹投放策略的参数信息

附录 2

介绍：该代码是某某语言编写的，作用是什么

```

import math
import matplotlib.pyplot as plt
import numpy as np

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 导弹位置函数，t 是时间
def M(t):
    # 导弹朝(0,0,0)飞，xz 平面里的参数
    tan_t = 2000 / 20000 # 初始 z 比 x
    c1 = math.sqrt(101) # 斜边
    sin_t = 1 / c1 # z 方向速度系数
    cos_t = 10 / c1 # x 方向速度系数
    v_m = 300 # 导弹速度 300m/s
    m_x = 20000 - v_m * t * cos_t # x 越来越小
    m_y = 0 # y 不变
    m_z = 2000 - v_m * t * sin_t # z 越来越小
    return [m_x, m_y, m_z]

# 烟幕位置函数，分情况算
def S(t):
    g1 = 9.8 # 重力

```

```

v_d = 120          # 无人机速度
t1 = 1.5           # 投放时间
t2 = t1 + 3.6      # 起爆时间
v_s = 3            # 起爆后下沉速度

# 无人机位置，z 固定 1800
d_x = 17800 - v_d * t
d_y = 0
d_z = 1800

if t < t1:
    # 没投放，跟着无人机
    return [d_x, d_y, d_z]
elif t1 <= t < t2:
    # 投放了没起爆，自由下落
    dt = t - t1    # 过了多久
    s_x = d_x      # x 和无人机一样
    s_y = 0
    s_z = 1800 - 0.5 * g1 * (dt ** 2) # 下落公式
    return [s_x, s_y, s_z]
else:
    # 起爆了，匀速下沉
    dt1 = t2 - t1  # 投放到期起爆的时间
    s_x2 = 17800 - v_d * t2
    s_y2 = 0
    s_z2 = 1800 - 0.5 * g1 * (dt1 ** 2)
    # 起爆后过了多久
    dt2 = t - t2
    s_z = s_z2 - v_s * dt2
    return [s_x2, s_y2, s_z]

# 真目标的点，生成一些
def get_true_target_points(n=12):
    p = []
    # 上下底面圆心
    p.append([0, 200, 0])
    p.append([0, 200, 10])
    # 下底面圆周的点
    ths = [2 * math.pi * i / n for i in range(n)]
    for th in ths:
        x = 7 * math.cos(th)
        y = 200 + 7 * math.sin(th)
        z = 0
        p.append([x, y, z])

```

```

# 上底面圆周的点
for th in ths:
    x = 7 * math.cos(th)
    y = 200 + 7 * math.sin(th)
    z = 10
    p.append([x, y, z])
return p

# 算 P 点, lis 是参数
def P(t, lis):
    m1_x, m1_y, m1_z = M(t)
    o1x, o1y, o1z = 0, 200, 0    # 下底面圆心
    o2x, o2y, o2z = 0, 200, 10   # 上底面圆心
    r1 = 7    # 半径

    # 确定用哪个底面
    if lis in [1, 2]:
        cx, cy, cz = o1x, o1y, o1z
    else:
        cx, cy, cz = o2x, o2y, o2z

    # 水平方向的向量
    dx = m1_x - cx
    dy = m1_y - cy

    d_sq = dx**2 + dy**2
    r_sq = r1**2

    # 算两个系数
    f1 = r_sq / d_sq
    f2 = r1 * math.sqrt(d_sq - r_sq) / d_sq
    # 不同 lis 返回不同点
    if lis == 1:
        return (cx + f1 * dx - f2 * dy, cy + f1 * dy + f2 * dx, cz)
    elif lis == 2:
        return (cx + f1 * dx + f2 * dy, cy + f1 * dy - f2 * dx, cz)
    elif lis == 3:
        return (cx + f1 * dx - f2 * dy, cy + f1 * dy + f2 * dx, cz)
    elif lis == 4:
        return (cx + f1 * dx + f2 * dy, cy + f1 * dy - f2 * dx, cz)

# 算 Q 点, lis 是参数
def Q(t, lis):
    m1_x, m1_y, m1_z = M(t)

```

```

    olx, oly, olz = 0, 200, 0
    o2x, o2y, o2z = 0, 200, 10
    r1 = 7
    sin_s = m1_x / math.sqrt(m1_x**2 + 40000)
    cos_s = 200 / math.sqrt(m1_x**2 + 40000)
    if lis == 1:
        return r1*sin_s, 200 - r1*cos_s, 0
    if lis == 2:
        return (-1)*r1*sin_s, 200 + r1*cos_s, 0
    if lis == 3:
        return r1*sin_s, 200 - r1*cos_s, 10
    if lis == 4:
        return (-1)*r1*sin_s, 200 + r1*cos_s, 10

# 判断线段 MP 和球（烟幕 S，半径 R）交不交
def segment_sphere_intersection(M, P, S, R):
    mp_vec = [P[0]-M[0], P[1]-M[1], P[2]-M[2]] # MP 向量
    ms_vec = [S[0]-M[0], S[1]-M[1], S[2]-M[2]] # MS 向量

    # 避免 M 和 P 重合
    mp_sq = sum(mp**2 for mp in mp_vec)
    if mp_sq < 1e-10:
        return math.sqrt(sum(ms**2 for ms in ms_vec)) <= R + 1e-10

    # 算 t 值，在 0 到 1 之间
    t_val = sum(mp*ms for mp, ms in zip(mp_vec, ms_vec)) / mp_sq
    t_c = max(0.0, min(1.0, t_val)) # 限制范围

    # 最近的点 Q
    q_p = [M[0]+t_c*mp_vec[0], M[1]+t_c*mp_vec[1], M[2]+t_c*mp_vec[2]]

    # 算距离，判断是否相交
    qs_sq = sum((q-s)**2 for q, s in zip(q_p, S))
    dist.append(math.sqrt(qs_sq))
    return qs_sq <= (R + 1e-10)**2 # 加个小值防误差

# 算遮蔽时间
def calculate_occlusion_time():
    # 真目标点
    tp = get_true_target_points(n=100)

    # 时间范围：起爆后 20 秒
    t_s = 1.5 + 3.6 # 起爆时间
    t_e = t_s + 20 # 结束时间

```

```

t_list = np.linspace(t_s, t_e, 10000) # 很多时间点

occlist = []
cs = None # 当前区间起点

for t in t_list:
    # 看当前时间有没有遮蔽
    m_p = M(t)
    s_p = S(t)
    is_ok = False

    # 生成目标点
    tp = [P(t, i) for i in range(1, 5)] + [Q(t, i) for i in range(1, 5)]
    # 检查每个点
    for p in tp:
        if segment_sphere_intersection(m_p, p, s_p, 10):
            is_ok = True
            break
    # 记录最大距离和时间
    if max(dist) <= 10 or t < 9.6:
        dist_max.append(max(dist))
        tim.append(t)
    if max(dist) <= 10 :
        dist_2.append(max(dist))
        tim_2.append(t)
    dist.clear()
    # 更新区间
    if is_ok:
        if cs is None:
            cs = t # 开始新区间
        else:
            if cs is not None:
                # 结束区间
                occlist.append((cs, t))
                cs = None

# 处理最后一个区间
if cs is not None:
    occlist.append((cs, t_e))

# 总时间
total_t = sum(end - start for start, end in occlist)
return occlist, total_t

```



```

# 主程序
if __name__ == "__main__":
    # 定义一些列表存数据
    dist = []
    dist_max = []
    tim = []
    tim_2 = []
    dist_2 = []
    # 算遮蔽时间
    occlist, total_t = calculate_occlusion_time()
    total_t = max(tim_2) - min(tim_2)
    # 输出结果
    print("烟幕对导弹的遮蔽结果")
    print(f'起爆时间： {1.5+3.6:.1f}s ， 时间范围： {1.5+3.6:.1f}s ~ {1.5+3.6+20:.1f}s')
    print("\n 遮蔽区间： ")
    for i, (s, e) in enumerate(occlist, 1):
        print(f' 第{i}个： {min(tim_2):.4f}s ~ {max(tim_2):.4f}s ， 时长： {max(tim_2)-min(tim_2):.4f}s')
    print(f"\n 总遮蔽时间： {total_t:.4f}秒")

```

附录 2

介绍： P2.py 求解第 2 题

```

import numpy as np
import math
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from deap import base, creator, tools, algorithms
import random
import time
from multiprocessing import Pool, cpu_count
from functools import partial
import numba
from numba import jit

# 字体设置，不然中文乱码
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 各种常量

```

```

zl = 9.8 # 重力
dds = 300 # 导弹速度
ytxs = 3 # 烟幕下沉速度
ytbj = 10 # 烟幕半径
ytcs = 20 # 烟幕持续时间
ts = 0.01 # 时间步长

# 初始位置
wjs = np.array([17800, 0, 1800]) # 无人机起点
ddq = np.array([20000, 0, 2000]) # 导弹起点
zdb = np.array([0, 200, 0]) # 真目标下面圆心
jd = np.array([0, 0, 0]) # 假目标
zdbj = 7 # 真目标半径
zgdg = 10 # 真目标高度

# 导弹朝假目标飞的方向
ddfx = jd - ddq
ddfx_norm = ddfx / np.linalg.norm(ddfx)
ddv = dds * ddfx_norm # 导弹速度向量

# 算导弹飞到目标要多久，投放时间不能超过这个
def jsgjls(ddwz, mubiao):
    """算导弹到目标时间"""
    jl = np.linalg.norm(mubiao - ddwz)
    return jl / dds
max_tf = jsgjls(ddq, jd) # 最大投放时间，大概 67 秒

# 记优化过程的东西
ls = []
kst = time.time() # 开始时间

def jhjd(a):
    """角度归一到 0 到 2  $\pi$  """
    return a % (2 * np.pi)

# 预存导弹位置，省得每次算
ddts = np.linspace(0, max_tf, 1000) # 导弹时间采样
ddwzs = np.array([ddq + ddv * t for t in ddts]) # 导弹位置采样

def ddwz(t):
    """快速查导弹位置，用之前存的插值"""
    idx = np.searchsorted(ddts, t)
    if idx == 0:

```

```

        return ddwzs[0]
    elif idx == len(ddts):
        return ddwzs[-1]
    else:
        # 线性插值，随便起的变量名
        t0, t1 = ddts[idx - 1], ddts[idx]
        d0, d1 = ddwzs[idx - 1], ddwzs[idx]
        b = (t - t0) / (t1 - t0)
        return d0 + b * (d1 - d0)

def szdb():
    """生成真目标表面的点，方便判断遮蔽"""
    d = []

    # 上下底面圆心
    d.append(zdb) # 下底面
    d.append(zdb + np.array([0, 0, zdgd])) # 上底面

    # 上下底面圆周 8 个点
    jdls = np.linspace(0, 2 * np.pi, 8, endpoint=False)
    for j in jdls:
        # 下底面圆周
        x = zdbj * np.cos(j)
        y = zdbj * np.sin(j)
        d.append(zdb + np.array([x, y, 0]))

        # 上底面圆周
        d.append(zdb + np.array([x, y, zdgd]))

    # 侧面 3 层，每层 8 个点
    gd = [zdgd / 4, zdgd / 2, 3 * zdgd / 4]
    for h in gd:
        for j in jdls:
            x = zdbj * np.cos(j)
            y = zdbj * np.sin(j)
            d.append(zdb + np.array([x, y, h]))

    return np.array(d)

# 真目标的点存起来
zdb = szdb()

# 加速判断烟幕有没有用，numba 装饰器不能删
@jit(nopython=True)

```

```

def pdyt(ytzx, ddwz, zdbz):
    """判断烟幕能不能挡住导弹看目标"""
    # 每个目标点都查一遍
    for i in range(zdbz.shape[0]):
        d = zdbz[i]
        # 导弹到目标点的向量
        ddzd = d - ddwz
        ddzdl = np.linalg.norm(ddzd)

        if ddzdl < 1e-6: # 防止除以 0
            if np.linalg.norm(ytzx - d) <= ytbj:
                return True
            continue

        # 单位方向向量
        f = ddzd / ddzdl

        # 导弹到烟幕中心的向量
        ddyt = ytzx - ddwz

        # 投影长度
        t = np.dot(ddyt, f)

        # 判断垂足在哪
        if t < 0:
            # 垂足在导弹后面，看导弹在不在烟幕里
            if np.linalg.norm(ddyt) <= ytbj:
                return True
        elif t > ddzdl:
            # 垂足在目标后面，看目标点在不在烟幕里
            if np.linalg.norm(ytzx - d) <= ytbj:
                return True
        else:
            # 垂足在中间，算距离
            z = ddwz + t * f
            ytz = ytzx - z
            if np.linalg.norm(ytz) <= ytbj:
                return True

    return False

def pgs(g, zdbz, max_tf):
    """包装函数，并行用的"""

```

```

return pg(g, zdbs, max_tf)

def pg(g, zdbs, max_tf):
    """算这个个体好不好（有效时间）"""
    wsd, jd, tf, sc = g # 无人机速度、方向、投放时间、投放起爆间隔

    # 超范围就不好，返回 0
    if wsd < 70 or wsd > 140 or tf < 0 or tf > max_tf or sc < 0.1:
        return 0,

    # 角度归一，更新一下
    jd = jhjd(jd)
    g[1] = jd

    qb = tf + sc # 起爆时间

    # 算投放点位置
    wsfx = np.array([np.cos(jd), np.sin(jd), 0])
    tfwz = wjs + wsd * wsfx * tf

    # 算起爆点位置（烟幕弹自由下落）
    qbwz = tfwz.copy()
    qbwz[2] -= 0.5 * zl * sc ** 2 # 垂直下落

    # 起爆点不能在地下
    if qbwz[2] < 0:
        return 0,

    # 起爆太晚导弹都飞过去了，没用
    if qb > max_tf:
        return 0,

    # 烟幕落到地面的时间
    dmt = qb + qbwz[2] / ytxs

    # 烟幕有效时间窗口
    ytjs = min(qb + ytc, dmt, max_tf)

    # 有效时间计数
    yxsj = 0

    # 采样时间点
    bs = max(1, int((ytjs - qb) / ts))
    sjd = np.linspace(qb, ytjs, bs)

```

```

# 先算好这些时间的导弹位置
ddwz_ls = [ddwz(t) for t in sjd]

# 每个时间点查一下
for i, t_current in enumerate(sjd):
    # 当前烟幕位置（下沉）
    ytwz = qbwz.copy()
    dt = t_current - qb
    ytwz[2] -= ytxs * dt

    # 落地了就不算了
    if ytwz[2] < 0:
        break

    # 拿之前算好的导弹位置
    ddwz_current = ddwz_ls[i]

    # 判断有没有用，有用就加时间
    if pdyt(ytwz, ddwz_current, zdb):
        yxsj += ts
return yxsj,

def by(g, mu, sigma, indpb):
    """自定义变异"""
    # 高斯变异
    for i in range(len(g)):
        if random.random() < indpb:
            g[i] += random.gauss(mu, sigma)

    # 角度归一
    g[1] = jhjd(g[1])

    # 其他变量限范围
    g[0] = max(70, min(140, g[0])) # 速度
    g[2] = max(0, min(max_tf, g[2])) # 投放时间
    g[3] = max(0.1, min(19.1, g[3])) # 投放起爆间隔

    return g,

def jlzj(qk):
    """记一下当前最好的个体"""
    if not qk:

```

```

        return

    # 找最好的
    zjg = max(qk, key=lambda x: x.fitness.values[0])
    wsd, jd, tf, sc = zjg
    yxsj = zjg.fitness.values[0]

    # 存到历史里
    ls.append({
        'wsd': wsd,
        'jd': jd,
        'tf': tf,
        'sc': sc,
        'yxsj': yxsj,
        'time': time.time() - ks
    })

```

附录 4

介绍：P3.py 求解第 3 题

```

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # 画 3D 用的
from scipy.optimize import differential_evolution
import pandas as pd
import time
import warnings
import openpyxl

warnings.filterwarnings("ignore")

# ----- 能改的参数 -----
dt_step = 0.01 # 时间一步多大，调试时可以改 0.02 或 0.05
t_max = 100    # 最大时间到多少
work_num = 1   # 并行的个数，原代码是 1
rand_seed = 0  # 随机种子，固定这个结果一样
pop_num = 80   # 种群大小，原代码 80
max_iter = 40  # 最多迭代多少次
# -----

# 设随机种子
np.random.seed(rand_seed)

# 固定的数，别动

```

```

g = 9.8          # 重力, m/s²
vm = 300         # 导弹速度
R_cloud = 10     # 烟幕半径
v_sink = 3       # 烟幕往下掉的速度
cloud_life = 20 # 烟幕能用多久
cyl_r = 7        # 目标圆柱半径
cyl_h = 10       # 目标圆柱高度

# 一开始的位置
missile_start = np.array([20000.0, 0.0, 2000.0]) # 导弹起始点
drone_start = np.array([17800.0, 0.0, 1800.0])   # 无人机起始点
target_O = np.array([0.0, 0.0, 0.0])            # 假目标 O
target_base = np.array([0.0, 200.0, 0.0])        # 真目标底面中心

# 导弹朝假目标 O 飞的方向 (单位向量)
drone_dir_missile = (target_O - missile_start) / np.linalg.norm(target_O -
missile_start)

# 时间列表, 从 0 到 t_max, 每次加 dt_step
time_list = np.arange(0.0, t_max + dt_step, dt_step)
time_len = len(time_list)
# 算导弹大概飞多久到真目标
est_fly_time = np.linalg.norm(missile_start - target_base) / vm
print(f'导弹大概飞多久: {est_fly_time:.2f} 秒')

# 做目标的关键点, 比如底面中心、顶面中心这些
def get_key_points(base=target_base, radius=cyl_r, height=cyl_h, n_points=8):
    points = []
    points.append(base.copy()) # 底面圆心
    points.append(base + np.array([0.0, 0.0, height])) # 顶面圆心
    # 底面周边点
    for i in range(n_points):
        ang = 2 * np.pi * i / n_points
        points.append(base + radius * np.array([np.cos(ang), np.sin(ang), 0.0]))
    # 顶面周边点
    for i in range(n_points):
        ang = 2 * np.pi * i / n_points
        points.append(base + radius * np.array([np.cos(ang), np.sin(ang), 0.0]) +
np.array([0.0, 0.0, height]))
    return np.array(points)

# 得到关键点
points = get_key_points()

```



```

# ----- 判断线段和球交不交 -----
def line_sphere_intersect(p1, p2, c, R):
    # p1p2 线段, c 是球心, R 是半径
    # 端点在球里就算交
    if np.linalg.norm(p1 - c) <= R or np.linalg.norm(p2 - c) <= R:
        return True
    v = p2 - p1
    a = np.dot(v, v)
    if a == 0: # 线段是个点
        return False
    b = 2 * np.dot(p1 - c, v)
    c_val = np.dot(p1 - c, p1 - c) - R ** 2 # 避免和球心 c 重名
    disc = b * b - 4 * a * c_val
    if disc < 0: # 没交点
        return False
    sqrt_disc = np.sqrt(disc)
    t1 = (-b + sqrt_disc) / (2 * a)
    t2 = (-b - sqrt_disc) / (2 * a)
    # 看 t 在不在 0 到 1 之间
    return (0 <= t1 <= 1) or (0 <= t2 <= 1)

# ----- 算单个烟幕的遮蔽时间 -----
def single_flare_shield(param, missile_start, drone_dir_missile, vm, drone_start, g,
R_cloud, v_sink, cloud_life, points, time_list, dt_step):
    # param 里是角度、速度、投放时间、延迟时间
    angle, speed, t_release, t_delay = param
    t_boom = t_release + t_delay # 起爆时间
    # 无人机方向
    drone_dir = np.array([np.cos(angle), np.sin(angle), 0.0])
    # 投放点位置
    drone_rel_pos = drone_start + speed * t_release * drone_dir
    # 起爆点位置 (延迟时间里会下落)
    boom_pos = drone_rel_pos + t_delay * speed * drone_dir - 0.5 * g * t_delay **
2 * np.array([0.0, 0.0, 1.0])

    # 标记哪些时间是有效的
    is_good = np.zeros(len(time_list), dtype=bool)
    for i, t in enumerate(time_list):
        # 导弹当前位置
        missile_pos = missile_start + vm * t * drone_dir_missile
        # 烟幕在有效期内
        if t_boom <= t <= t_boom + cloud_life:
            # 烟幕当前位置 (往下掉)
            cloud_pos = boom_pos - v_sink * (t - t_boom) * np.array([0.0, 0.0,

```

```

1.0))
    # 所有关键点都被遮才算
    all_hide = True
    for kp in points:
        if not line_sphere_intersect(missile_pos, kp, cloud_pos,
R_cloud):
            all_hide = False
            break
    is_good[i] = all_hide

# 找连续的有效时间区间
good_idx = np.where(is_good)[0]
if good_idx.size == 0:
    return 0.0, [] # 没有有效时间
time_ranges = []
start = good_idx[0]
prev = good_idx[0]
for k in good_idx[1:]:
    if k - prev > 1: # 断了，保存之前的区间
        time_ranges.append((time_list[start], time_list[prev]))
        start = k
    prev = k
time_ranges.append((time_list[start], time_list[prev])) # 最后一个区间
# 算总时间
total_time = sum(end - start for start, end in time_ranges)
return total_time, time_ranges

# ----- 算多个烟幕的遮蔽时间 -----
def multi_flare_shield(param, missile_start, drone_dir_missile, vm, drone_start, g,
R_cloud, v_sink, cloud_life, points, time_list, dt_step):
    # param 里是角度、速度、基础投放时间、基础延迟、后面两个的增量
    angle, speed = param[0], param[1]
    # 三个烟幕的投放和延迟时间
    release_t = np.zeros(3)
    delay_t = np.zeros(3)
    release_t[0] = param[2]
    delay_t[0] = param[3]
    release_t[1] = release_t[0] + param[4]
    delay_t[1] = param[5]
    release_t[2] = release_t[1] + param[6]
    delay_t[2] = param[7]
    boom_t = release_t + delay_t # 三个烟幕的起爆时间
    # 无人机方向
    drone_dir = np.array([np.cos(angle), np.sin(angle), 0.0])

```

```

# 标记有效时间
is_good = np.zeros(len(time_list), dtype=bool)

# 先算三个烟幕的投放点和起爆点
rel_pos_list = np.zeros((3, 3))
boom_pos_list = np.zeros((3, 3))
for i in range(3):
    rel_pos_list[i, :] = drone_start + speed * release_t[i] * drone_dir
    boom_pos_list[i, :] = rel_pos_list[i, :] + delay_t[i] * speed * drone_dir -
0.5 * g * delay_t[i] ** 2 * np.array([0.0, 0.0, 1.0])

# 遍历每个时间点
for t_idx, t in enumerate(time_list):
    # 导弹当前位置
    missile_pos = missile_start + vm * t * drone_dir_missile
    # 收集当前存在的烟幕中心
    cloud_list = []
    for j in range(3):
        if boom_t[j] <= t <= boom_t[j] + cloud_life:
            cloud_pos = boom_pos_list[j, :] - v_sink * (t - boom_t[j]) *
np.array([0.0, 0.0, 1.0])
            cloud_list.append(cloud_pos)
    if not cloud_list: # 没烟幕, 跳过
        continue
    # 检查所有关键点是否被至少一个烟幕遮
    all_hide = True
    for kp in points:
        hide = False
        for cc in cloud_list:
            if line_sphere_intersect(missile_pos, kp, cc, R_cloud):
                hide = True
                break
        if not hide:
            all_hide = False
            break
    is_good[t_idx] = all_hide

# 算总有效时间
good_idx = np.where(is_good)[0]
if good_idx.size == 0:
    return 0.0
time_ranges = []
start = good_idx[0]

```

```

prev = good_idx[0]
for k in good_idx[1:]:
    if k - prev > 1:
        time_ranges.append((time_list[start], time_list[prev]))
        start = k
    prev = k
time_ranges.append((time_list[start], time_list[prev]))
total_time = sum(e - s for s, e in time_ranges)
return total_time

# 目标函数，因为优化是最小化，所以返回负的遮蔽时间（这样最小化负的就是最大化正的）
def eval_func(param):
    return -multi_flare_shield(
        param, missile_start, drone_dir_missile, vm, drone_start, g, R_cloud,
        v_sink, cloud_life, points, time_list, dt_step
    )

# ----- 主函数，跑起来的部分 -----
def main():
    # 参数的上下界
    low_bnd = [0.0, 70.0, 0.0, 0.1, 1.0, 0.1, 1.0, 0.1]
    up_bnd = [0.5, 140.0, 5.0, 3.0, 10.0, 3.0, 20.0, 3.0]
    bnd_list = list(zip(low_bnd, up_bnd))

    # 优化的参数设置
    opt_params = {
        'func': eval_func,
        'bounds': bnd_list,
        'maxiter': max_iter,
        'popsize': pop_num,
        'tol': 1e-7,
        'mutation': (0.5, 1),
        'recombination': 0.8,
        'strategy': 'best1bin',
        'disp': True,
        'polish': True,
        'workers': work_num
    }

    print("开始优化...")
    start_t = time.time()
    res = differential_evolution(**opt_params) # 跑优化
    end_t = time.time()

```

```

print(f'优化用了多久: {end_t - start_t:.2f} s')

# 最优参数
best_param = res.x
best_val = res.fun

# 拆最优参数
best_angle = best_param[0]
best_speed = best_param[1]
best_release_t = np.zeros(3)
best_delay_t = np.zeros(3)
best_release_t[0] = best_param[2]
best_delay_t[0] = best_param[3]
best_release_t[1] = best_release_t[0] + best_param[4]
best_delay_t[1] = best_param[5]
best_release_t[2] = best_release_t[1] + best_param[6]
best_delay_t[2] = best_param[7]
best_boom_t = best_release_t + best_delay_t # 最优起爆时间

# 角度转成度
best_angle_deg = best_angle * 180.0 / np.pi
# 总有效时间（负的 best_val）
total_good_time = -best_val

# 打印结果
print('\n 最好的参数:')
print(f' 无人机方向 (度): {best_angle_deg:.2f}')
print(f' 无人机速度 (m/s): {best_speed:.2f}')
print(f' 总共遮蔽时间 (s): {total_good_time:.2f}')

# 主函数
if __name__ == '__main__':
    main()
    qbwz_opt[2] -= 0.5 * zl * sc_opt ** 2

```

附录 4

介绍：P4.py 求解第 4 题

```

import numpy as np
import math
import matplotlib.pyplot as plt

```

```

from mpl_toolkits.mplot3d import Axes3D
from deap import base, creator, tools, algorithms
import random
import time
import pandas as pd
from openpyxl import Workbook
from multiprocessing import Pool, cpu_count
from functools import partial
import numba
from numba import jit

# 中文显示
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 各种常数
gravity = 9.8 # 重力
mis_spd = 300 # 导弹速度
cloud_down = 3 # 烟幕往下掉的速度
cloud_r = 10 # 烟幕半径
cloud_use_time = 20 # 烟幕能用多久
t_step = 0.01 # 时间一步多长

# 开始的位置
mis_start = np.array([20000, 0, 2000]) # 导弹开始的地方
real_tgt_cen = np.array([0, 200, 0]) # 真目标下面的中心
fake_tgt = np.array([0, 0, 0]) # 假目标
tgt_r = 7 # 真目标半径
tgt_h = 10 # 真目标高度

# 无人机开始的位置
drone_pos = {
    'FY1': np.array([17800, 0, 1800]),
    'FY2': np.array([12000, 1400, 1400]),
    'FY3': np.array([6000, -3000, 700])
}

# 导弹朝假目标飞的方向
mis_dir = fake_tgt - mis_start
mis_dir_norm = mis_dir / np.linalg.norm(mis_dir)
mis_v = mis_spd * mis_dir_norm

# 算导弹飞到目标要多久

```

```

def calc_mis_time(mis_p, tgt_p):
    dist = np.linalg.norm(tgt_p - mis_p)
    return dist / mis_spd

max_t = calc_mis_time(mis_start, fake_tgt) # 大概 67 秒

# 记优化过程的东西
his = []
start_t = time.time()

# 角度弄成 0 到  $2\pi$  之间
def norm_ang(theta):
    return theta % (2 * np.pi)

# 算导弹在 t 时候的位置
def mis_pos(t):
    return mis_start + mis_v * t

# 判断烟幕有用没
@jit(nopython=True)
def check_cloud(cloud_cen, mis_p, tgt_pnts):
    # 看导弹到每个目标点的线挡没挡
    for i in range(tgt_pnts.shape[0]):
        p = tgt_pnts[i]
        mt = p - mis_p
        mt_norm = np.linalg.norm(mt)

        if mt_norm < 1e-6: # 剔除 0
            if np.linalg.norm(cloud_cen - p) <= cloud_r:
                return True
            continue

        u = mt / mt_norm
        mc = cloud_cen - mis_p
        l = np.dot(mc, u)

        if l < 0:
            # 垂足在导弹后面，看导弹在不在烟幕里
            if np.linalg.norm(mc) <= cloud_r:
                return True

```

```

        elif l > mt_norm:
            # 垂足在目标后面，看目标点在不在烟幕里
            if np.linalg.norm(cloud_cen - p) <= cloud_r:
                return True
        else:
            # 垂足在线段上，算距离
            h = mis_p + l * u
            ch = cloud_cen - h
            if np.linalg.norm(ch) <= cloud_r:
                return True

    return False

# 生成真目标的点
def gen_tgt_pnts():
    pnts = []

    # 上下底面中心
    pnts.append(real_tgt_cen) # 下面
    pnts.append(real_tgt_cen + np.array([0, 0, tgt_h])) # 上面

    # 上下底面圆周 8 个点
    angles = np.linspace(0, 2 * np.pi, 8, endpoint=False)
    for a in angles:
        dx = tgt_r * np.cos(a)
        dy = tgt_r * np.sin(a)
        pnts.append(real_tgt_cen + np.array([dx, dy, 0]))
        pnts.append(real_tgt_cen + np.array([dx, dy, tgt_h]))

    # 侧面 3 层，每层 8 个点
    hs = [tgt_h / 4, tgt_h / 2, 3 * tgt_h / 4]
    for h in hs:
        for a in angles:
            dx = tgt_r * np.cos(a)
            dy = tgt_r * np.sin(a)
            pnts.append(real_tgt_cen + np.array([dx, dy, h]))

    return np.array(pnts)

# 生成目标点
tgt_pnts_np = np.array(gen_tgt_pnts())

```



```

# 预先算导弹位置，后面插值用
mis_t_samples = np.linspace(0, max_t, 1000)
mis_p_samples = np.array([mis_pos(t) for t in mis_t_samples])

# 快一点算导弹位置
def mis_pos_fast(t):
    idx = np.searchsorted(mis_t_samples, t)
    if idx == 0:
        return mis_p_samples[0]
    elif idx == len(mis_t_samples):
        return mis_p_samples[-1]
    else:
        t0, t1 = mis_t_samples[idx - 1], mis_t_samples[idx]
        p0, p1 = mis_p_samples[idx - 1], mis_p_samples[idx]
        alpha = (t - t0) / (t1 - t0)
        return p0 + alpha * (p1 - p0)

# 评估个体好不好
def eval_ind(ind, tgt_pts_np, drone_pos, max_t):
    # 拆参数，3个无人机各4个
    p1 = ind[0:4] # 速度、角度、投放时间、间隔
    p2 = ind[4:8]
    p3 = ind[8:12]

    # 检查参数范围
    for p in [p1, p2, p3]:
        v, ang, t_drop, delta = p
        if v < 70 or v > 140 or t_drop < 0 or t_drop > max_t or delta < 0.1:
            return 0, [0, 0, 0]

    # 无人机参数打包
    drone_params = [
        (p1, drone_pos['FY1'], 'FY1'),
        (p2, drone_pos['FY2'], 'FY2'),
        (p3, drone_pos['FY3'], 'FY3')
    ]

    cloud_wins = [] # 烟幕有效时间
    drone_clouds = {} # 无人机的烟幕信息

    for p, start_p, d_id in drone_params:
        v, ang, t_drop, delta = p

```

```

# 角度归一
ang = norm_ang(ang)

t_explode = t_drop + delta # 起爆时间

# 算投放点
drone_dir = np.array([np.cos(ang), np.sin(ang), 0])
drop_p = start_p + v * drone_dir * t_drop

# 算起爆点，只掉高度
explode_p = drop_p.copy()
explode_p[2] -= 0.5 * gravity * delta ** 2

# 起爆点不能在地下
if explode_p[2] < 0:
    drone_clouds[d_id] = None
    continue

# 起爆太晚没用
if t_explode > max_t:
    drone_clouds[d_id] = None
    continue

# 烟幕落地时间
t_ground = t_explode + explode_p[2] / cloud_down

# 烟幕有效结束时间
t_win_end = min(t_explode + cloud_use_time, t_ground, max_t)

cloud_info = (explode_p, t_explode, t_win_end, d_id)
cloud_wins.append(cloud_info)
drone_clouds[d_id] = cloud_info

# 没有有效烟幕就返回 0
if not cloud_wins:
    return 0, [0, 0, 0]

# 算总有效时间和每个无人机的时间
total_t = 0
drone_t = {'FY1': 0, 'FY2': 0, 'FY3': 0}

# 收集时间点，别重复
all_t = set()

```

```

for _, t_s, t_e, _ in cloud_wins:
    step_num = max(1, int((t_e - t_s) / t_step))
    t_points = np.linspace(t_s, t_e, step_num)
    all_t.update(t_points)

# 排序时间点
t_sorted = sorted(all_t)

# 预先算导弹位置
mis_pos_list = [mis_pos_fast(t) for t in t_sorted]

# 检查每个时间点
for i, t_now in enumerate(t_sorted):
    masked = False
    mask_drones = set()

    for explode_p, t_ex, t_we, d_id in cloud_wins:
        # 烟幕现在有效吗
        if t_now < t_ex or t_now > t_we:
            continue

        # 算当前烟幕位置
        cloud_p = explode_p.copy()
        dt = t_now - t_ex
        cloud_p[2] -= cloud_down * dt

        # 落地了就跳过
        if cloud_p[2] < 0:
            continue

        # 拿导弹位置
        mis_p = mis_pos_list[i]

        # 看挡不挡
        if check_cloud(cloud_p, mis_p, tgt_pnts_np):
            masked = True
            mask_drones.add(d_id)

    if masked:
        total_t += t_step
        # 给无人机加时间
        for d_id in mask_drones:
            drone_t[d_id] += t_step

```

```

    return total_t, [drone_t['FY1'], drone_t['FY2'], drone_t['FY3']]

# 包装评估函数，并行用
def eval_ind_wrap(ind, tgt_pnts_np, drone_pos, max_t):
    return eval_ind(ind, tgt_pnts_np, drone_pos, max_t)

# DEAP 用的评估函数
def eval_ind_deap(ind):
    total_t, _ = eval_ind(ind, tgt_pnts_np, drone_pos, max_t)
    return total_t,

# 自己写的变异函数
def my_mutate(ind, mu, sigma, indpb):
    # 高斯变异
    for i in range(len(ind)):
        if random.random() < indpb:
            ind[i] += random.gauss(mu, sigma)

    # 角度归一
    for i in range(0, len(ind), 4):
        if i + 1 < len(ind):
            ind[i + 1] = norm_ang(ind[i + 1])

    # 其他参数范围
    for i in range(0, len(ind), 4):
        # 速度
        ind[i] = max(70, min(140, ind[i]))
        # 投放时间
        if i + 2 < len(ind):
            ind[i + 2] = max(0, min(max_t, ind[i + 2]))
        # 间隔
        if i + 3 < len(ind):
            ind[i + 3] = max(0.1, min(19.1, ind[i + 3]))

    return ind,

# 记最好的个体
def save_best(pop):
    if not pop:
        return

```

```

# 找最好的
best = max(pop, key=lambda x: x.fitness.values[0])
eff_t = best.fitness.values[0]

# 拆参数
p1 = best[0:4]
p2 = best[4:8]
p3 = best[8:12]

```

附录 4

介绍：P4.py 求解第 4 题

```

import numpy as np
import math
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from deap import base, creator, tools, algorithms
import random
import time
from openpyxl import Workbook
from multiprocessing import Pool, cpu_count
from numba import jit

# 中文显示
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 各种常量
zl = 9.8 # 重力加速度
dds = 300 # 导弹速度
ytx = 3 # 烟幕云团下沉速度
yf = 10 # 烟幕云团有效半径
ys = 20 # 烟幕有效持续时间
ts = 0.05 # 时间步长

# 初始位置
m1_s = np.array([20000, 0, 2000])
m2_s = np.array([19000, 600, 2100])
m3_s = np.array([18000, -600, 1900])
z1 = np.array([0, 200, 0]) # 真目标圆柱下底面圆心

```

```

z2 = np.array([0, 0, 0])    # 假目标
zr = 7    # 目标半径
zh = 10 # 目标高度

# 无人机初始位置，用个字典存
wjr = {
    'FY1': np.array([17800, 0, 1800]),
    'FY2': np.array([12000, 1400, 1400]),
    'FY3': np.array([6000, -3000, 700]),
    'FY4': np.array([11000, 2000, 1800]),
    'FY5': np.array([13000, -2000, 1300])
}

# 算导弹速度方向的函数
def dd_fx(s_p):
    dd_f = z2 - s_p    # 导弹朝向假目标
    dd_fz = dd_f / np.linalg.norm(dd_f)    # 归一化
    return dds * dd_fz    # 乘以速度大小

# 三个导弹的速度
dd1_fx = dd_fx(m1_s)
dd2_fx = dd_fx(m2_s)
dd3_fx = dd_fx(m3_s)

# 算导弹飞到目标的时间
def dd_sj(d_p, t_p):
    juli = np.linalg.norm(t_p - d_p)    # 距离
    return juli / dds    # 时间=距离/速度

# 每个导弹的最大投放时间
max_t1 = dd_sj(m1_s, z2)
max_t2 = dd_sj(m2_s, z2)
max_t3 = dd_sj(m3_s, z2)
max_t = max(max_t1, max_t2, max_t3)    # 取最大的那个

# 算导弹在某个时间的位置
def dd_wz(t, d_s, d_f):
    return d_s + d_f * t    # 位置=初始+速度*时间

# 做导弹位置表
def dd_wz_biao(d_s, d_f, max_time, num_samples=1000):
    t_biao = np.linspace(0, max_time, num_samples)    # 时间点
    w_biao = np.array([dd_wz(t, d_s, d_f) for t in t_biao])    # 对应位置
    return t_biao, w_biao

```

```

# 三个导弹的位置表
m1_t_biao, m1_w_biao = dd_wz_biao(m1_s, dd1_fx, max_t)
m2_t_biao, m2_w_biao = dd_wz_biao(m2_s, dd2_fx, max_t)
m3_t_biao, m3_w_biao = dd_wz_biao(m3_s, dd3_fx, max_t)

# 快速查导弹位置的函数
def dd_wz_kuai(t, d_id):
    # 选对应的导弹表
    if d_id == 1:
        t_b, w_b = m1_t_biao, m1_w_biao
    elif d_id == 2:
        t_b, w_b = m2_t_biao, m2_w_biao
    else:
        t_b, w_b = m3_t_biao, m3_w_biao

    idx = np.searchsorted(t_b, t) # 找时间对应的索引
    if idx == 0:
        return w_b[0]
    elif idx == len(t_b):
        return w_b[-1]
    else:
        # 插值算位置
        t0, t1 = t_b[idx-1], t_b[idx]
        p0, p1 = w_b[idx-1], w_b[idx]
        alpha = (t - t0) / (t1 - t0)
        return p0 + alpha * (p1 - p0)

# 判断烟幕有没有用
@jit(nopython=True)
def yz_youxiao(y_z, d_p, t_d):
    for i in range(t_d.shape[0]):
        dian = t_d[i]
        m_t = dian - d_p # 导弹到目标点的向量
        m_tz = np.linalg.norm(m_t)
        if m_tz < 1e-6: # 距离太近
            if np.linalg.norm(y_z - dian) <= yf:
                return True
            continue
        u = m_t / m_tz # 单位向量
        m_y = y_z - d_p # 导弹到烟幕中心
        l = np.dot(m_y, u) # 投影长度
        if l < 0: # 烟幕在导弹后面
            if np.linalg.norm(m_y) <= yf:

```

```

        return True
    elif l > m_tz: # 烟幕在目标点后面
        if np.linalg.norm(y_z - dian) <= yf:
            return True
    else: # 烟幕在中间，算垂距
        H = d_p + l * u
        CH = y_z - H
        if np.linalg.norm(CH) <= yf:
            return True
    return False

# 生成目标的采样点
def z_dian():
    dian = []
    dian.append(z1) # 下底面圆心
    dian.append(z1 + np.array([0, 0, zh])) # 上底面圆心

    # 底面和顶面的圆周点，8 个角度
    jiao = np.linspace(0, 2*np.pi, 8, endpoint=False)
    for j in jiao:
        dx = zr * np.cos(j)
        dy = zr * np.sin(j)
        dian.append(z1 + np.array([dx, dy, 0])) # 底面圆周
        dian.append(z1 + np.array([dx, dy, zh])) # 顶面圆周

    return np.array(dian)

z_dian_np = np.array(z_dian()) # 转成 numpy 数组

# 角度归一化，防止超过 360 度
def jiao_zheng(j):
    return j % (2*np.pi)

# 评估个体的函数，算遮蔽时间
def pingjia(ge, z_dian_np, wjr, max_t):
    wjj = {} # 存每个无人机的参数
    for i, wj_id in enumerate(['FY1', 'FY2', 'FY3', 'FY4', 'FY5']):
        start_idx = i * 8 # 每个无人机占 8 个参数
        w_jd = ge[start_idx] # 无人机速度
        w_j = ge[start_idx + 1] # 无人机方向角
        t_fang = [ge[start_idx + 2], ge[start_idx + 4], ge[start_idx + 6]] # 投放
        t_q = [ge[start_idx + 3], ge[start_idx + 5], ge[start_idx + 7]] # 延迟

```

时间

时间


```

wjj[wj_id] = {'v': w_jd, 'j': w_j, 't_f': t_fang, 't_q': t_q}

y_wind = [] # 烟幕的有效窗口
for wj_id, cs in wjj.items():
    w_jd = cs['v']
    w_j = cs['j']
    s_p = wjr[wj_id] # 无人机初始位置
    w_fx = np.array([np.cos(w_j), np.sin(w_j), 0]) # 无人机方向
    # 循环三次投放
    for i, (t_f, t_qj) in enumerate(zip(cs['t_f'], cs['t_q'])):
        # 检查参数合不合理
        if not (0 <= t_f <= max_t and 70 <= w_jd <= 140 and t_qj >= 0.1):
            continue
        t_qb = t_f + t_qj # 起爆时间
        f_wz = s_p + w_jd * w_fx * t_f # 投放位置
        # 起爆位置：水平继续动，垂直下落
        q_wz = f_wz + w_jd * w_fx * t_qj
        q_wz[2] -= 0.5 * z1 * t_qj**2
        # 起爆位置太低或者时间超了就跳过
        if q_wz[2] < 0 or t_qb > max_t:
            continue
        # 烟幕落到地面的时间
        t_di = t_qb + q_wz[2] / ytx
        # 烟幕有效结束时间
        t_end = min(t_qb + ys, t_di, max_t)
        y_wind.append({'explode_pos': q_wz, 't_explode': t_qb,
't_window_end': t_end})
    # 没有有效烟幕就返回 0
    if not y_wind:
        return 0, {'M1':0,'M2':0,'M3':0}

dd_yt = {'M1':0,'M2':0,'M3':0} # 每个导弹的有效遮蔽时间
# 循环三个导弹
for d_id, d_max_t in zip([1,2,3],[max_t1,max_t2,max_t3]):
    t_dian = []
    # 收集所有有效时间点
    for c in y_wind:
        t_s = c['t_explode']
        t_e = min(c['t_window_end'], d_max_t)
        if t_s < t_e:
            t_dian.append(np.arange(t_s, t_e, ts))
    if not t_dian: # 没找到时间点就下一个导弹
        continue
    # 合并去重

```

```

t_all = np.unique(np.concatenate(t_dian))
# 算这些时间点的导弹位置
d_wz = np.array([dd_wz_kuai(t, d_id) for t in t_all])
# 检查每个时间点
for i, t_now in enumerate(t_all):
    d_p = d_wz[i]
    for c in y_wind:
        # 烟幕在有效时间内
        if c['t_explode'] <= t_now <= c['t_window_end']:
            y_z = c['explode_pos'].copy()
            # 烟幕下沉后的位置
            y_z[2] -= ytx * (t_now - c['t_explode'])
            # 判断有没有用
            if y_z[2] > 0 and yz_youxiao(y_z, d_p, z_dian_np):
                dd_yt[fM{d_id}] += ts
                break # 一个烟幕有用就够了
z_yt = sum(dd_yt.values()) # 总有效时间
return z_yt, dd_yt

# DEAP 要用的评估函数，只返回总时间
def pingjia_deap(ge):
    z_yt, _ = pingjia(ge, z_dian_np, wjr, max_t)
    return z_yt,

# 自定义变异函数，改参数用
def bianyi(ge, mu=0, sigma=0.5, indpb=0.2):
    # 每个参数都有概率变异
    for i in range(len(ge)):
        if random.random() < indpb:
            ge[i] += random.gauss(mu, sigma)
    # 变异后修正参数范围
    for i in range(0, len(ge), 8):
        ge[i+1] = jiao_zheng(ge[i+1]) # 角度归一化
        ge[i] = max(70, min(140, ge[i])) # 速度范围
        # 投放时间要递增
        t1 = max(0, min(max_t, ge[i+2]))
        t2 = max(t1+1, min(max_t, ge[i+4]))
        t3 = max(t2+1, min(max_t, ge[i+6]))
        ge[i+2], ge[i+4], ge[i+6] = t1, t2, t3
        # 延迟时间范围
        ge[i+3] = max(0.1, min(19.1, ge[i+3]))
        ge[i+5] = max(0.1, min(19.1, ge[i+5]))
        ge[i+7] = max(0.1, min(19.1, ge[i+7]))
    return ge,

```

```

# 主程序开始
if __name__ == '__main__':
    # 创建 DEAP 需要的类
    creator.create("FitnessMax", base.Fitness, weights=(1.0,)) # 最大化适应度
    creator.create("Individual", list, fitness=creator.FitnessMax) # 个体是列表
    toolbox = base.Toolbox()

    # 初始化个体的函数
    def chushihua():
        ge = []
        # 5 个无人机, 每个 8 个参数
        for _ in range(5):
            ge.append(random.uniform(70,140)) # 速度
            ge.append(random.uniform(0,2*np.pi)) # 方向角
            # 投放时间递增
            t1 = random.uniform(0, max_t)
            t2 = random.uniform(t1+1, max_t)
            t3 = random.uniform(t2+1, max_t)
            ge.append(t1); ge.append(random.uniform(0.1,19.1)) # t1 和延迟 1
            ge.append(t2); ge.append(random.uniform(0.1,19.1)) # t2 和延迟 2
            ge.append(t3); ge.append(random.uniform(0.1,19.1)) # t3 和延迟 3
        return ge

    # 注册各种工具函数
    toolbox.register("individual", tools.initIterate, creator.Individual, chushihua)
    toolbox.register("population", tools.initRepeat, list, toolbox.individual)
    toolbox.register("evaluate", pingjia_deap) # 评估
    toolbox.register("mate", tools.cxBlend, alpha=0.5) # 交叉
    toolbox.register("mutate", bianyid, mu=0, sigma=0.5, indpb=0.2) # 变异
    toolbox.register("select", tools.selTournament, tournsize=3) # 选择

    # 创建种群, 200 个个体
    pop = toolbox.population(n=200)
    # 保存最好的个体
    hof = tools.HallOfFame(1)
    # 多进程加速
    pool = Pool(processes=cpu_count())
    toolbox.register("map", pool.map)

    # 统计数据
    stats = tools.Statistics(lambda ind: ind.fitness.values)
    stats.register("avg", np.mean)
    stats.register("min", np.min)

```

```
stats.register("max", np.max)

# 运行遗传算法，200 代
pop, log = algorithms.eaSimple(pop, toolbox, cxpb=0.8, mutpb=0.2,
                               ngen=200, stats=stats, halloffame=hof,
verbose=True)
pool.close() # 关闭进程池

# 取最好的个体算结果
zuihao = hof[0]
zuihao_yt, dd_yt = pingjia(zuihao, z_dian_np, wjr, max_t)
```